Web Mining and Recommender Systems

Temporal data mining: Regression for Sequence Data

This topic

Temporal models

This topic will look back on some of the topics already covered in this class, and see how they can be adapted to make use of **temporal** information

1. **Regression** – sliding windows and autoregression
2. **Recommender systems** – temporal recommendation and the Netflix Prize
3. Sequential Recommendation

Previously – Regression

Given **labeled training data** of the form

$$\{(\text{data}_1, \text{label}_1), \ldots, (\text{data}_n, \text{label}_n)\}$$

Infer the function

$$f(\text{data}) \xrightarrow{?} \text{labels}$$

Time-series regression

Here, we'd like to predict sequences of **real-valued** events as accurately as possible.

Time-series regression

Here, we'd like to predict sequences of **real-valued** events as accurately as possible.

Given: a time series: $(x_1, \ldots, x_N) \in \mathbb{R}^N$

Suppose we'd like to minimize the MSE (as usual!) of the final part of some continuous portion of the sequence

$$\frac{1}{u-v+1} \sum_{t=u}^{v} (f_t(x_1, \ldots, x_{u-1}) - x_t)^2$$

Time-series regression

Method 1: maintain a "moving average" using a window of some fixed length

$$f(x_1, \ldots, x_m) =$$

Time-series regression

Method 1: maintain a "moving average" using a window of some fixed length

- This can be computed efficiently via dynamic programming:

$$f(x_1, \ldots, x_{m+1}) =$$

Time-series regression

Method 1: maintain a "moving average" using a window of some fixed length

$$f(x_1, \ldots, x_m) = \frac{1}{K} \sum_{k=0}^{K-1} x_{m-k}$$

- This can be computed efficiently via dynamic programming:

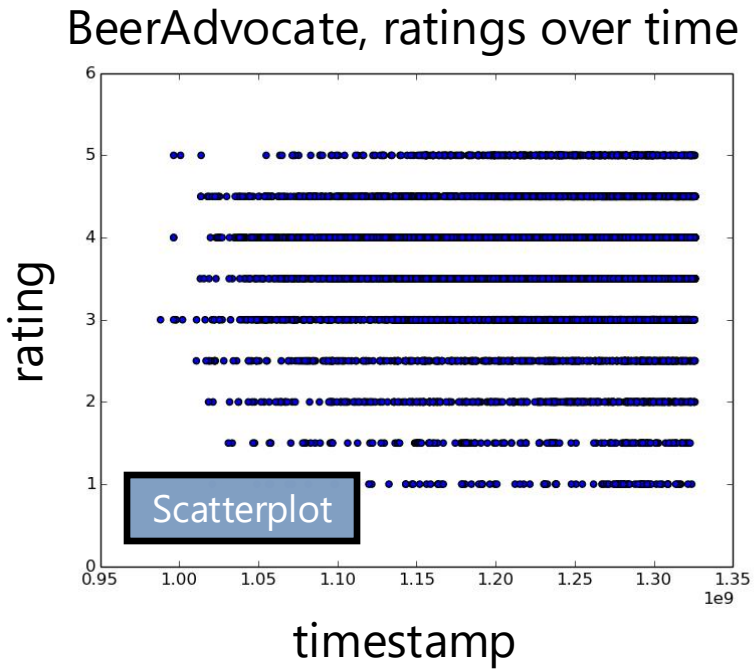
$$f(x_1, \ldots, x_{m+1}) = \frac{1}{K}(K \cdot f(x_1, \ldots, x_m) - x_{m-k} + x_{m+1})$$

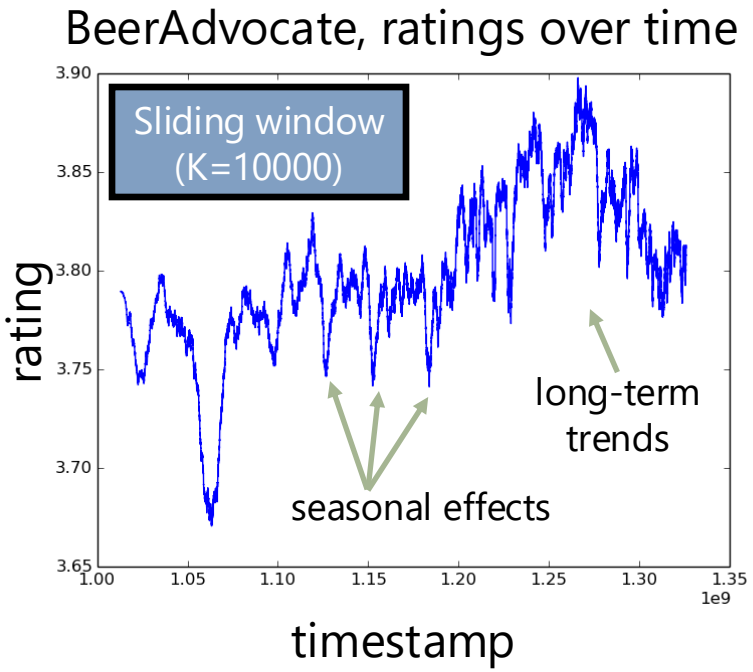
"peel-off" the oldest point

add the newest point

Time-series regression

Also useful to plot data:





Code on course webpage

Time-series regression

Method 2: weight the points in the moving average by age

$$f(x_1, \ldots, x_m) =$$

Time-series regression

Method 2: weight the points in the moving average by age

newest points have the highest weight

weight decays to zero after K points

$$f(x_1, \ldots, x_m) = \frac{\sum_{k=0}^{K-1} (K-k) x_{m-k}}{\binom{K}{2}}$$

Time-series regression

Method 3: weight the most recent points exponentially higher

$$f(x_1) =$$

$$f(x_1, \ldots, x_m) =$$

Methods 1, 2, 3

Method 1: Sliding window
Method 2: Linear decay
Method 3: Exponential decay

Time-series regression

Method 4: all of these models are assigning **weights** to previous values using some predefined scheme, why not just **learn** the weights?

$$f(x_1, \ldots, x_m) =$$

Time-series regression

Method 4: all of these models are assigning **weights** to previous values using some predefined scheme, why not just **learn** the weights?

- We can now fit this model using least-squares
- This procedure is known as **autoregression**
- Using this model, we can capture **periodic** effects, e.g. that the traffic of a website is most similar to its traffic 7 days ago

Web Mining and Recommender Systems

Temporal recommender systems: the Netflix Prize

Extensions of latent-factor models

Lots of reasons preferences could change over time…

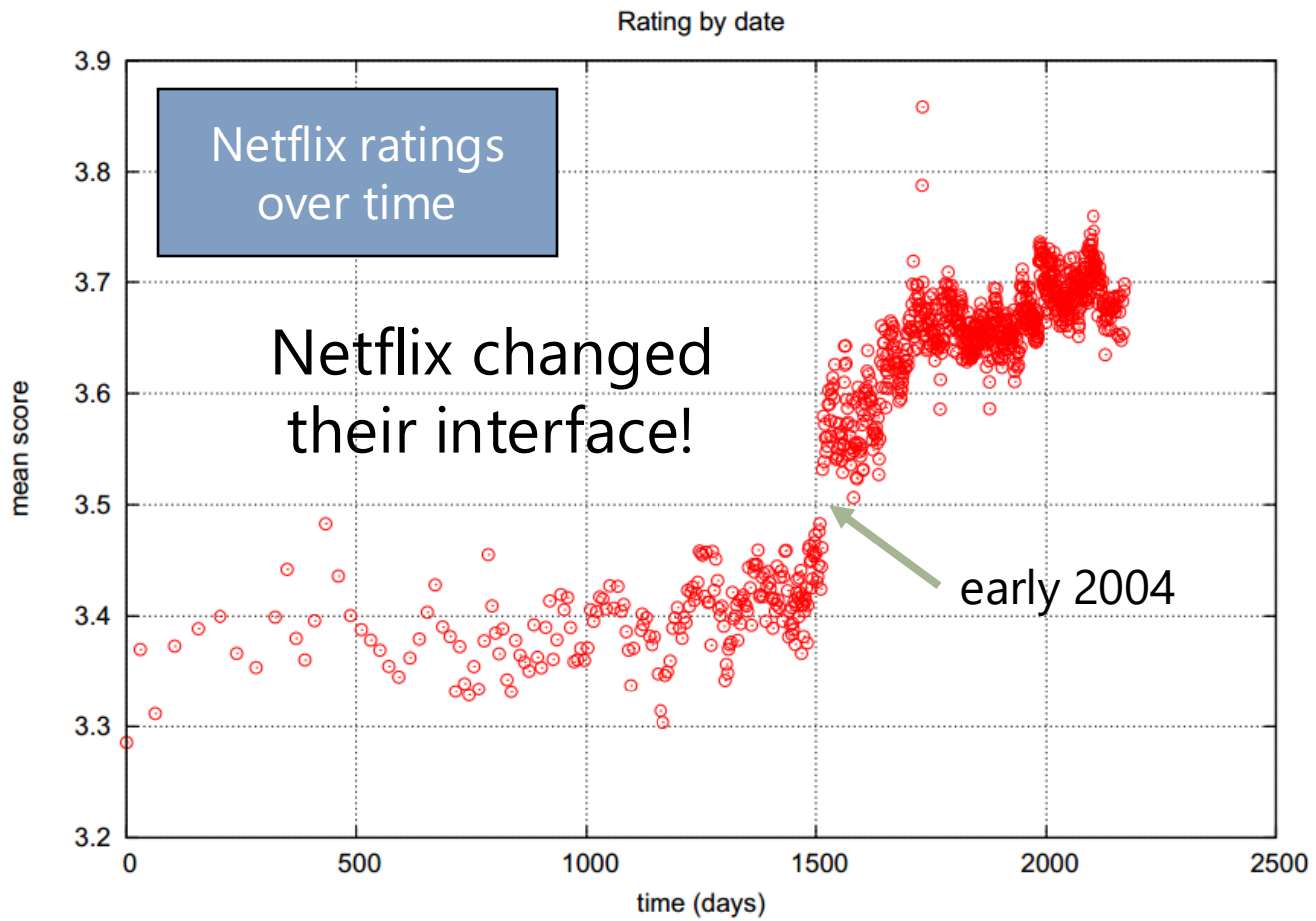


Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Extensions of latent-factor models

Lots of reasons preferences could change over time…

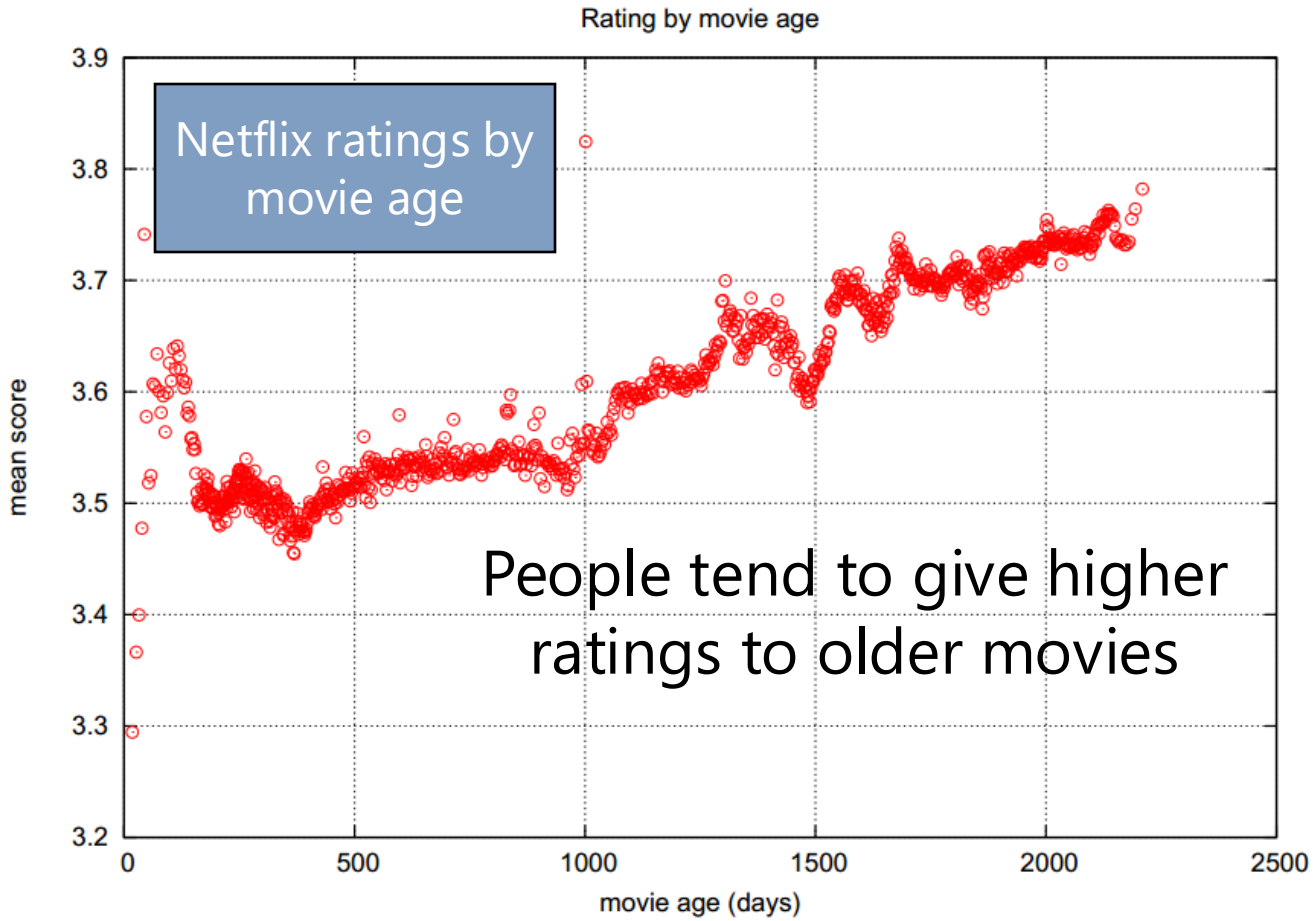
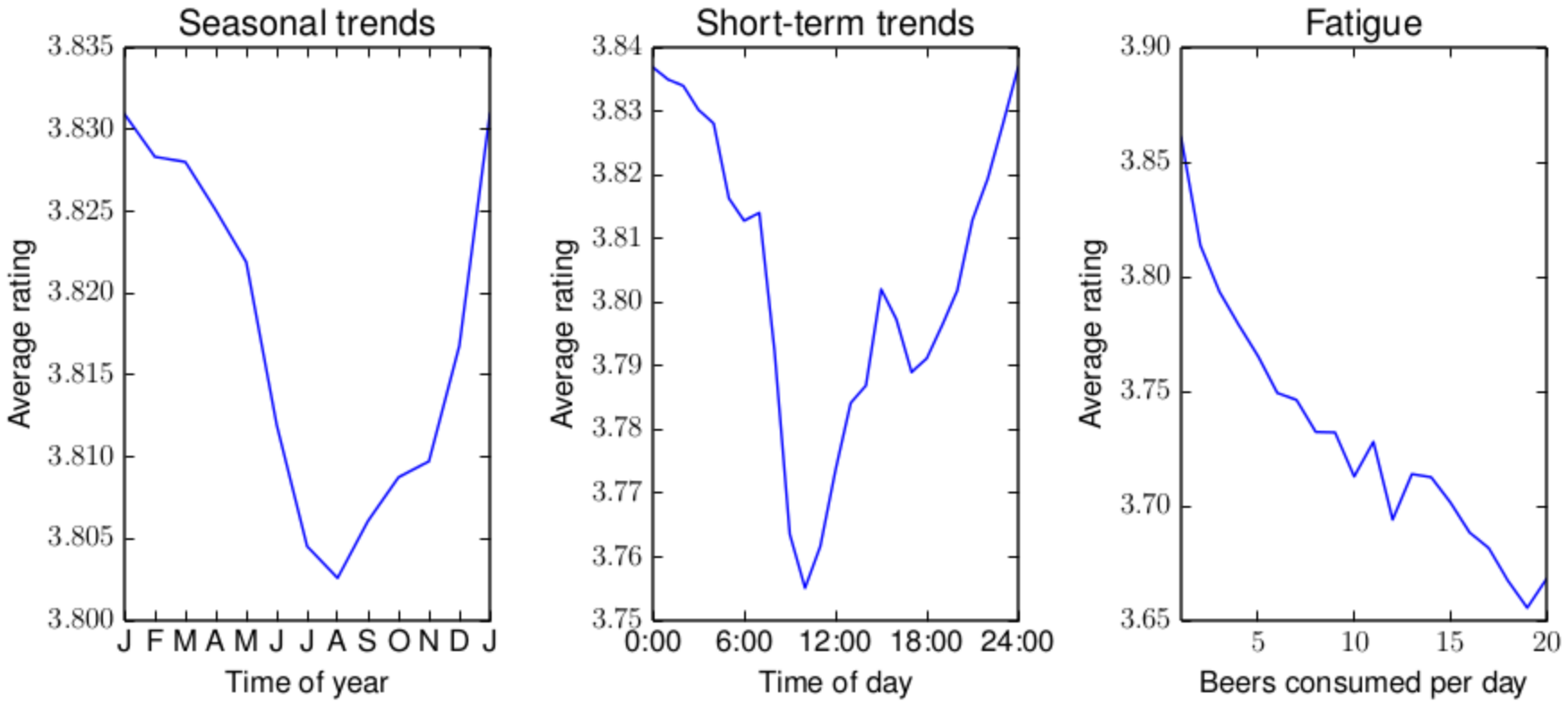


Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Extensions of latent-factor models

Lots of reasons preferences could change over time...

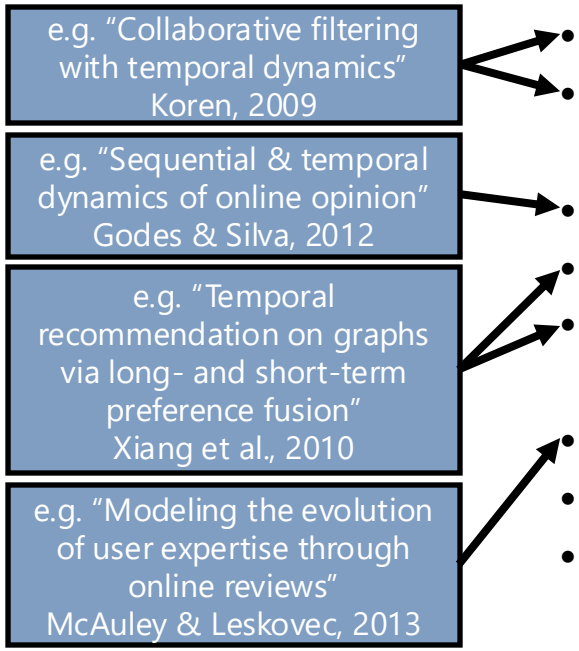


A few temporal effects from beer reviews

Extensions of latent-factor models

Lots of reasons preferences could change over time…

There are a number of reasons why rating data might be subject to temporal effects…



- Changes in the interface
- People give higher ratings to older movies (or, people who watch older movies are a biased sample)
- The community's preferences gradually change over time
- My girlfriend starts using my Netflix account one day
- I binge watch all 144 episodes of buffy one week and then revert to my normal behavior
- I become a "connoisseur" of a certain type of movie
- Anchoring, public perception, seasonal effects, etc.
- More in textbook!

Web Mining and Recommender Systems

Temporal recommender systems: simple heuristics

Simple temporal models

How can simple recommenders be adapted to handle temporal factors?

Previously we saw simple recommenders that predict ratings based on similarity functions:

Simple temporal models

How can this type of model be adjusted to account for temporal factors?

Simple temporal models

How can this type of model be adjusted to account for temporal factors?

Simple temporal models

How can this type of model be adjusted to account for temporal factors?

$$r(u,i) = \frac{\sum_{j \in I_u} R_{u,j} \cdot \mathrm{Sim}(i,j) \cdot f(t_{u,j})}{\sum_{j \in I_u} \mathrm{Sim}(i,j) \cdot f(t_{u,j})}$$

e.g. $f(t) = e^{-\lambda \cdot t}$

Simple temporal models

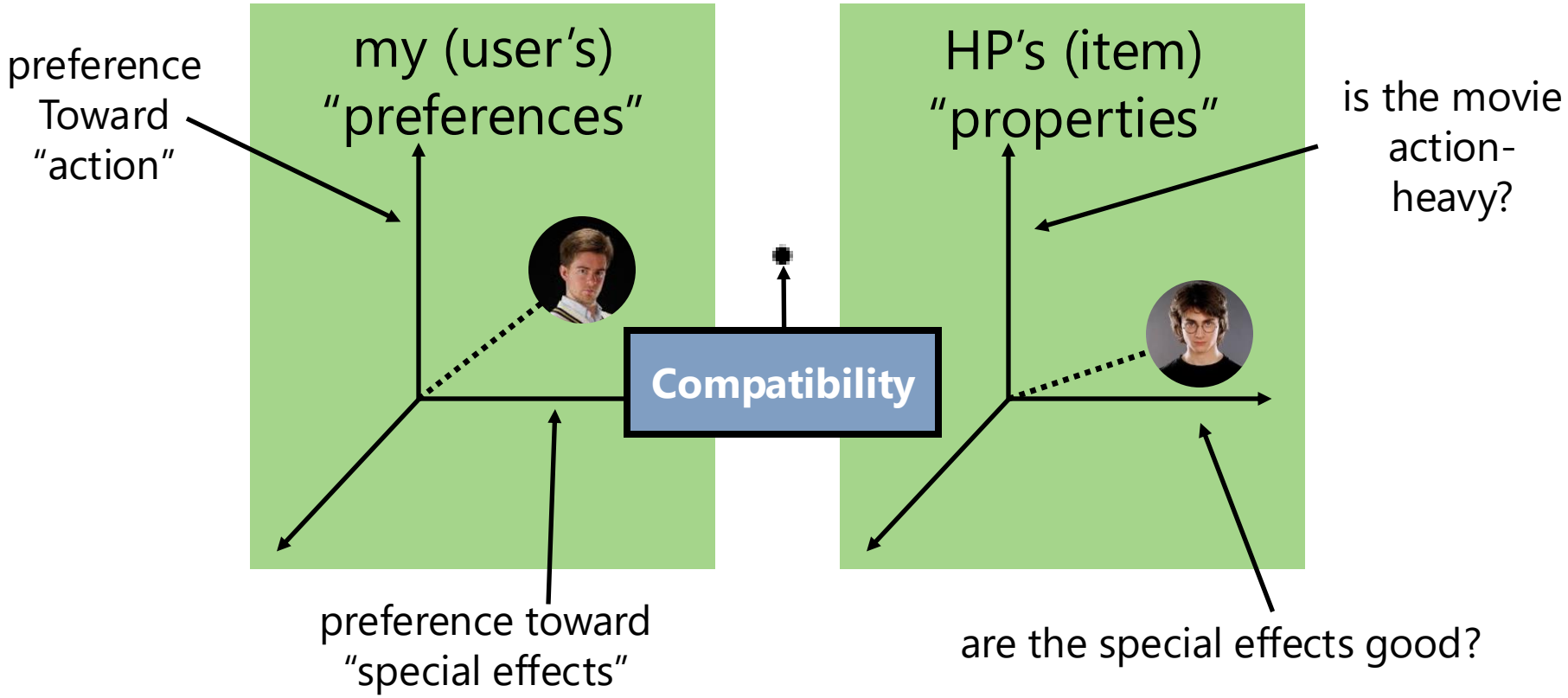
(see more in textbook)

Web Mining and Recommender Systems

Temporal recommender systems: the Netflix Prize

Previously…

Recommender Systems go beyond the methods we've seen so far by trying to model the **relationships** between people and the items they're evaluating



Previously…

Predict a user's rating of an item according to:

$$f(u,i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

By solving the optimization problem:

$$\arg\min_{\alpha,\beta,\gamma} \underbrace{\sum_{u,i}(\alpha+\beta_u+\beta_i+\gamma_u\cdot\gamma_i-R_{u,i})^2}_{\text{error}} + \underbrace{\lambda\left[\sum_u \beta_u^2 + \sum_i \beta_i^2 + \sum_i \|\gamma_i\|_2^2 + \sum_u \|\gamma_u\|_2^2\right]}_{\text{regularizer}}$$

(e.g. using stochastic gradient descent)

Extensions of latent-factor models

Different definitions of temporal evolution demand slightly different model assumptions (we'll see some in more detail later!) but the basic idea is the following:

Start with our original model:

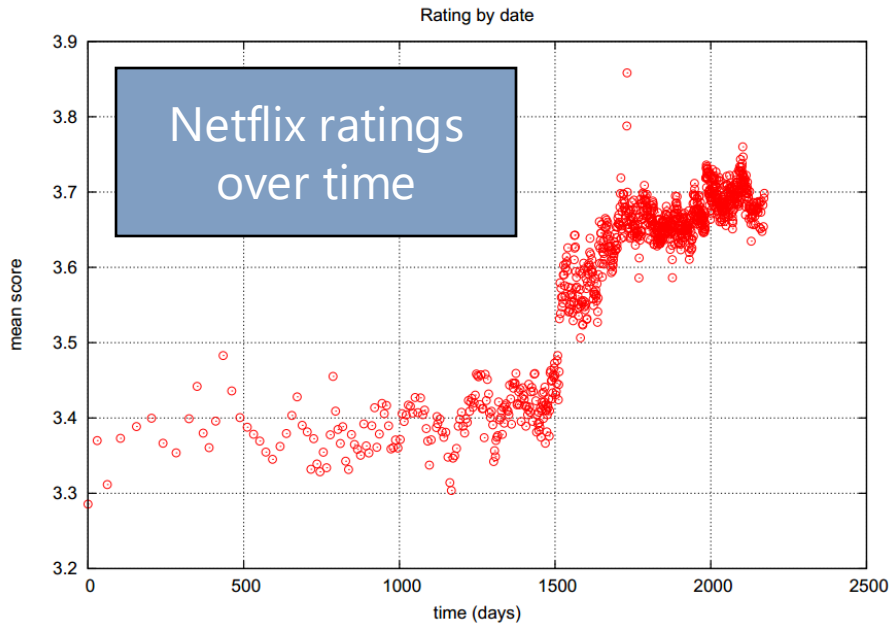
$$f(u,i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

And define some of the parameters as a function of time, e.g.:

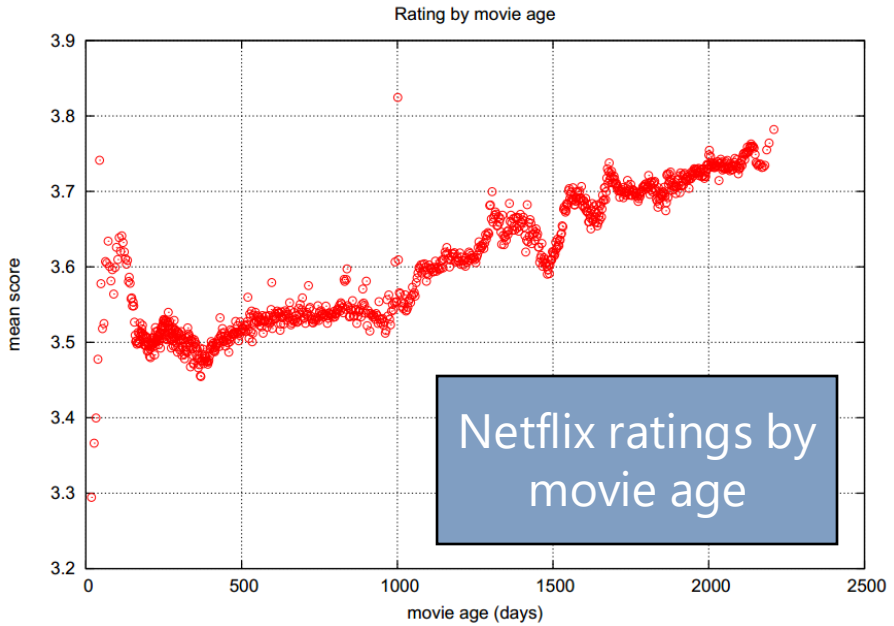
$$f(u,i,t) = \alpha + \beta_u(t) + \beta_i(t) + \gamma_u(t) \cdot \gamma_i$$

Temporal latent-factor models

To build a reliable system (and to win the Netflix prize!) we need to account for **temporal dynamics:**



(Netflix changed their interface)



(People tend to give higher ratings to older movies)

So how was this actually done?

Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Temporal latent-factor models

To start with, let's just assume that it's only the **bias** terms that explain these types of temporal variation (which, for the examples on the previous slides, is potentially enough)

$$b_{u,i}(t) = \alpha + \beta_u(t) + \beta_i(t)$$

Idea: temporal dynamics for *items* can be explained by long-term, gradual changes, whereas for users we'll need a different model that allows for "bursty", short-lived behavior

Temporal latent-factor models

temporal bias model:

$$b_{u,i}(t) = \alpha + \beta_u(t) + \beta_i(t)$$

For item terms, just separate the dataset into (equally sized) bins:*

$$\beta_i(t) = \beta_i + \beta_{i,\mathrm{Bin}(t)}$$

*in Koren's paper they suggested ~30 bins corresponding to about 10 weeks each for Netflix

or bins for periodic effects (e.g. the day of the week):

$$\beta_i(t) = \beta_i + \beta_{i,\mathrm{Bin}(t)} + \beta_{i,\mathrm{period}(t)}$$

What about user terms?

- We need something much finer-grained
- **But** – for most users we have far too little data to fit very short term dynamics

Temporal latent-factor models

Start with a simple model of drifting dynamics for users:

mean rating date for user u

hyperparameter (ended up as x=0.4 for Koren)

$$\mathrm{dev}_u(t) = \underbrace{\mathrm{sign}(t - t_u)}_{\text{before (-1) or after (1) the mean date}} \cdot \underbrace{|t - t_u|}_{\text{days away from mean date}}^{x}$$

Temporal latent-factor models

Start with a simple model of drifting dynamics for users:

mean rating date for user u

hyperparameter (ended up as x=0.4 for Koren)

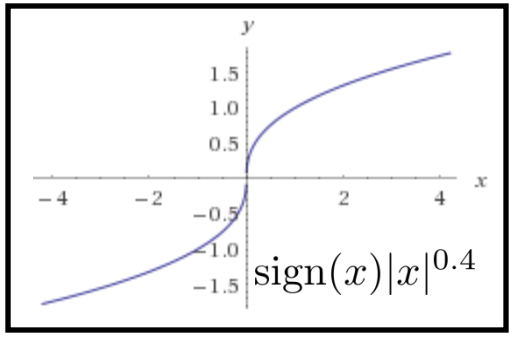
$$\mathrm{dev}_u(t) = \underbrace{\mathrm{sign}(t - t_u)}_{\text{before (-1) or after (1) the mean date}} \cdot \underbrace{|t - t_u|}_{\text{days away from mean date}}^{x}$$

time-dependent user bias can then be defined as:

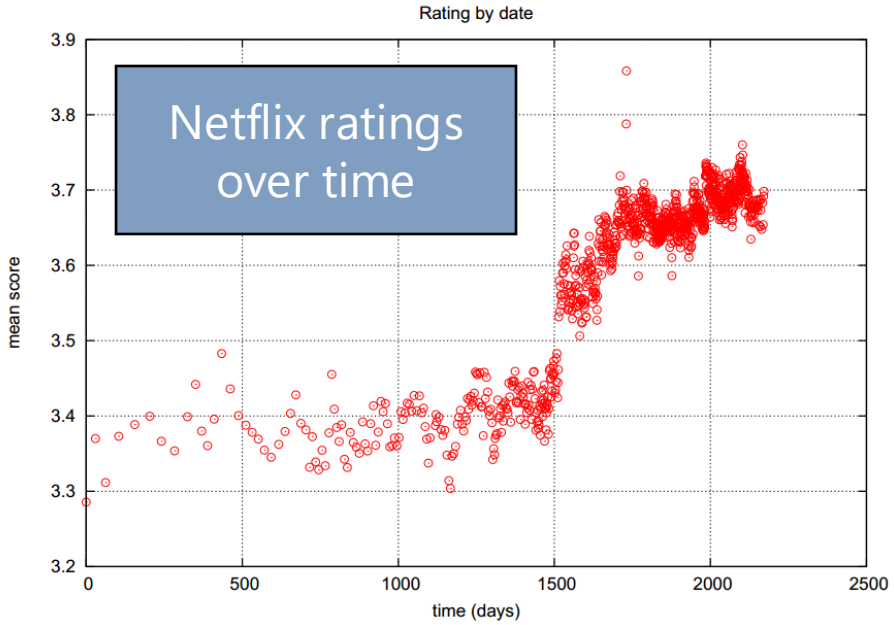
$$\beta_u^{(1)}(t) = \beta_u + \alpha_u \cdot \mathrm{dev}_u(t)$$

overall user bias

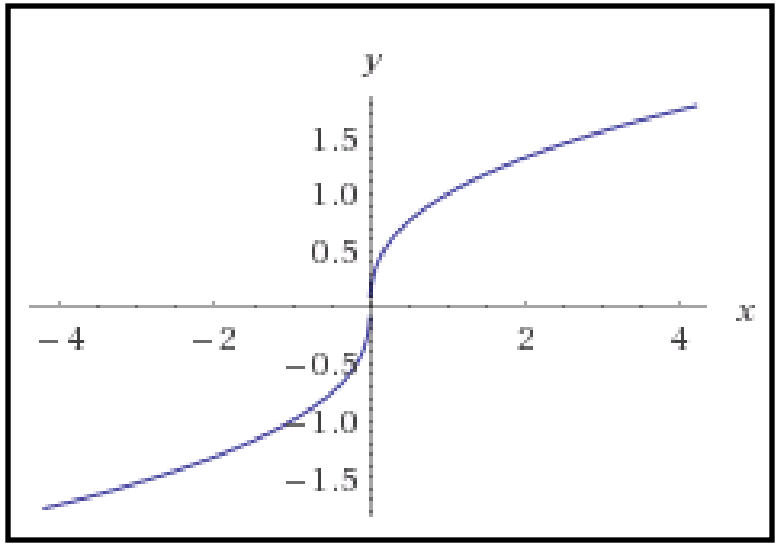
sign and scale for deviation term



Temporal latent-factor models



Real data



Fitted model

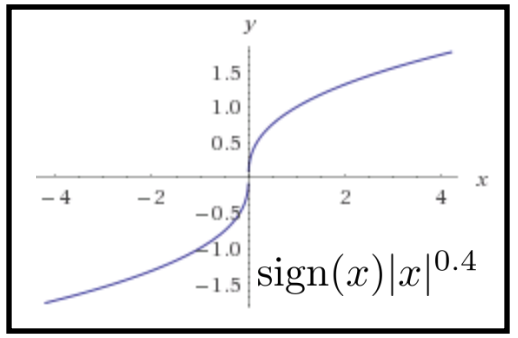
Temporal latent-factor models

time-dependent user bias can then be defined as:

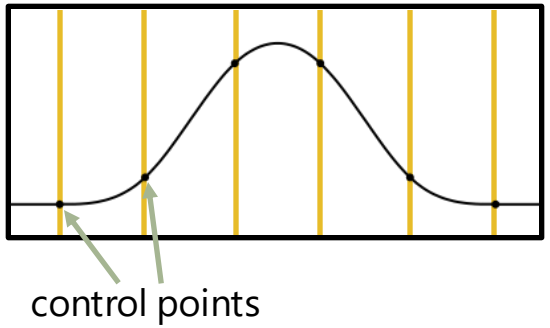
$$\beta_u^{(1)}(t) = \beta_u + \alpha_u \cdot \mathrm{dev}_u(t)$$

overall user bias

sign and scale for deviation term



- Requires only two parameters per user and captures some notion of temporal "drift" (even if the model found through cross-validation is (to me) completely unintuitive)
- To develop a slightly more expressive model, we can interpolate smoothly between biases using splines



Temporal latent-factor models

number of control points for this user
(k_u = n_u^0.25 in Koren)

user bias associated with this control point

$$\beta_u^{(2)}(t) = \beta_u + \frac{\sum_{l=1}^{k_u} e^{-\gamma|t-t_l^u|} b_{t_l}^u}{\sum_{l=1}^{k_u} e^{-\gamma|t-t_l^u|}}$$

time associated with control point
(uniformly spaced)

Temporal latent-factor models

number of control points for this user (k_u = n_u^0.25 in Koren)

user bias associated with this control point

$$\beta_u^{(2)}(t) = \beta_u + \frac{\sum_{l=1}^{k_u} e^{-\gamma|t-t_l^u|} b_{t_l}^u}{\sum_{l=1}^{k_u} e^{-\gamma|t-t_l^u|}}$$

time associated with control point (uniformly spaced)

- This is now a reasonably flexible model, but still only captures *gradual drift*, i.e., it can't handle sudden changes (e.g. a user simply having a bad day)

Temporal latent-factor models

- Koren got around this just by adding a "per-day" user bias:

$$\beta_{u,t}$$

bias for a particular day (or session)

- Of course, this is only useful for particular days in which users have a lot of (abnormal) activity
- The final (time-evolving bias) model then combines all of these factors:

$$\beta_{u,i}(t) = \alpha + \beta_u + \alpha_u \cdot \mathrm{dev}_u(t) + \beta_{u,t} + \beta_i + \beta_{i,\mathrm{Bin}(t)}$$

global offset

gradual deviation (or splines)

item bias

gradual item bias drift

user bias

single-day dynamics

Temporal latent-factor models

Finally, we can add a time-dependent scaling factor:

$$\beta_{u,i}(t) = \alpha + \beta_u + \alpha_u \cdot \mathrm{dev}_u(t) + \beta_{u,t} + (\beta_i + \beta_{i,\mathrm{Bin}(t)}) \cdot c_u(t)$$

also defined as $c_u + c_{u,t}$

Latent factors can also be defined to evolve in the same way:

$$\gamma_{u,k}(t) = \gamma_{u,k} + \alpha_{u,k} \cdot \mathrm{dev}_u(t) + \gamma_{u,k,t}$$

factor-dependent user drift

factor-dependent short-term effects

Moral(s) of the story

Summary

- Effective modeling of temporal factors was absolutely critical to this solution outperforming alternatives on Netflix's data
  - In fact, even with only temporally evolving *bias* terms, their solution was already ahead of Netflix's previous ("Cinematch") model

On the other hand…

  - Many of the ideas here depend on dynamics that are quite specific to "Netflix-like" settings
- Some factors (e.g. short-term effects) depend on a high density of data per-user and per-item, which is not always available

Moral(s) of the story

How much do these extension help?

Moral: increasing complexity helps a bit, but changing the model can help **a lot**

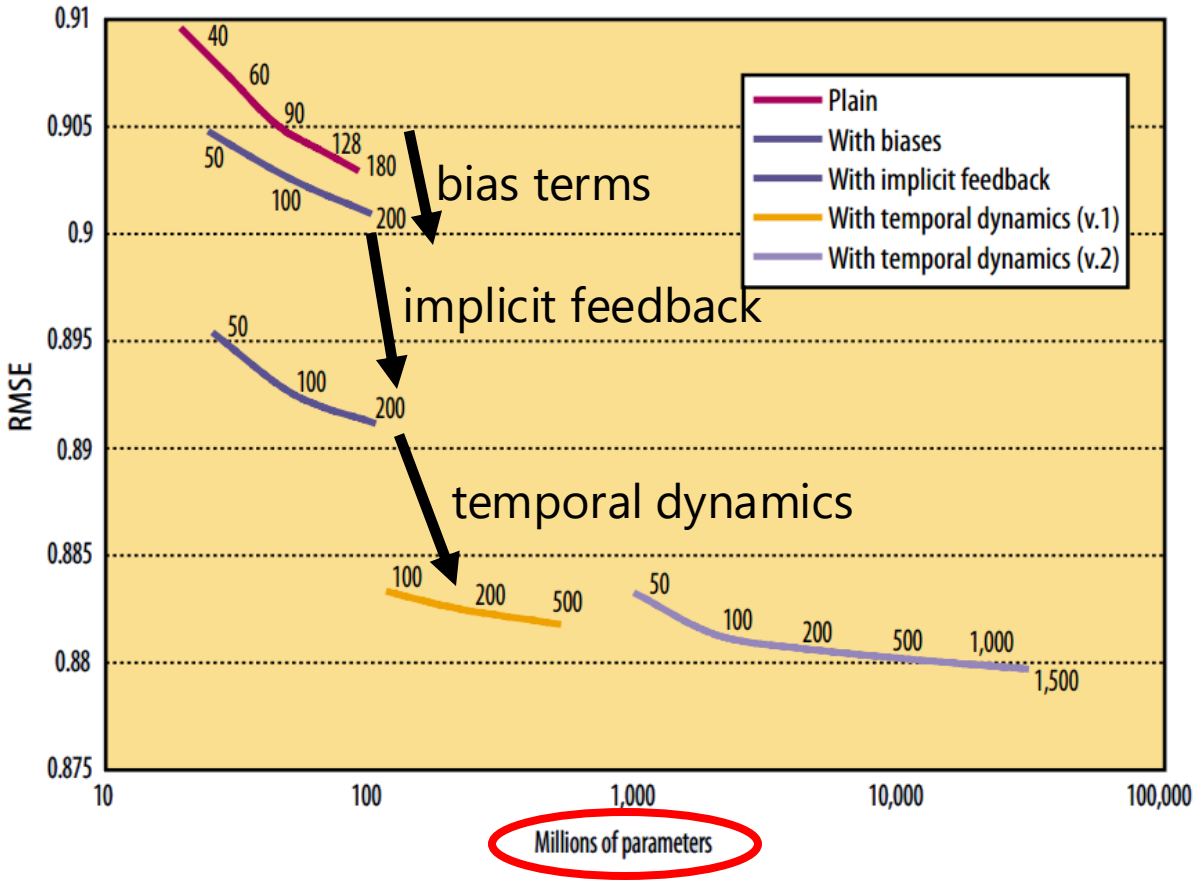


Figure from Koren: "Collaborative Filtering with Temporal Dynamics" (KDD 2009)

Moral(s) of the story

So what actually happened with Netflix?

- The AT&T team "BellKor", consisting of Yehuda Koren, Robert Bell, and Chris Volinsky were early leaders. Their main insight was how to effectively incorporate temporal dynamics into recommendation on Netflix.
- Before long, it was clear that no one team would build the winning solution, and Frankenstein efforts started to merge. Two frontrunners emerged, "BellKor's Pragmatic Chaos", and "The Ensemble".
- The BellKor team was the first to achieve a 10% improvement in RMSE, putting the competition in "last call" mode. The winner would be decided after 30 days.
- After 30 days, performance was evaluated on the hidden part of the test set.
- Both of the frontrunning teams had **the same** RMSE (up to some precision) but BellKor's team submitted their solution 20 minutes earlier and won $1,000,000

For a less rough summary, see the Wikipedia page about the Netflix prize, and the nytimes article about the competition: http://goo.gl/WNpy7o

Moral(s) of the story

Afterword

- Netflix had a class-action lawsuit filed against them after somebody de-anonymized the competition data
- $1,000,000 seems to be **incredibly cheap** for a company the size of Netflix in terms of the amount of research that was devoted to the task, and the potential benefit to Netflix of having their recommendation algorithm improved by 10%
- Other similar competitions have emerged, such as the Heritage Health Prize ($3,000,000 to predict the length of future hospital visits)

- But… the winning solution never made it into production at Netflix – it's a monolithic algorithm that is very expensive to update as new data comes in*

*source: a friend of mine told me and I have no actual evidence of this claim

References

Further reading:
"Collaborative filtering with temporal dynamics"
Yehuda Koren, 2009
http://research.yahoo.com/files/kdd-fp074-koren.pdf

Web Mining and Recommender Systems

Markov-Chain based recommendation

Temporal versus Sequential Recommendation

So far, our study of temporal recommendation has mostly made use of *timestamps*; timestamps (or features extracted from them) can tell us:

- Whether users like action movies on Fridays
- Whether older movies gradually get more popular
- Whether certain movies are popular during certain seasons
- (other than movies) whether restaurants are popular on certain occasions
- Etc.

Temporal versus Sequential Recommendation

But a timestamp *can't* tell us (for example):

- I'm likely to watch Harry Potter 2 because I just watched Harry Potter 1
- I'm likely to purchase a memory card because I just purchased a camera
- I'm likely to watch more action movies because I've recently been watching action movies

These dynamics depend on the *ordering* of actions, rather than the *absolute time* they occurred

Markov Chain models

This type of dynamic can be captured via a *Markov Chain*

Precisely: next action is conditionally independent of the interaction history given the previous action

Markov Chain models

This type of dynamic can be captured via a *Markov Chain*

Precisely: next action is conditionally independent of the interaction history given the previous action

$$p(i^{(t+1)} = i \mid i^{(t)} \ldots i^{(1)}) = p(i^{(t+1)} = i \mid i^{(t)})$$

next item

interaction history

previous item

Markov Chain models

Ideally, this type of dynamic should be personalized to a user, i.e.,

- The next action should match the user's preferences
- It should also be consistent with the previous item

$$p(i_u^{(t+1)} = i \mid i_u^{(t)} \dots i_u^1) = p(i_u^{(t+1)} = i \mid i_u^{(t)})$$

Markov Chain models

Most of the time we don't have to worry too much about probabilities and precise mathematical formalism: we just want to fit a function

score associated with the next interaction

$$f(\overbrace{u, i}^{} \mid \underbrace{i_u^{(t-1)}}_{}).$$

given the user's previous interaction

Markov Chain models

Challenge: whereas our previous models required modeling a UxI *matrix,* we must now model a UxIxI *tensor:*

Markov Chain models: FPMC

Idea: decompose the tensor into pairwise factors

Markov Chain models: FPMC

Idea: decompose the tensor into pairwise factors

$$f(i|u,j) = \underbrace{\gamma_u^{(ui)} \cdot \gamma_i^{(iu)}}_{f(i|u)} + \underbrace{\gamma_i^{(ij)} \cdot \gamma_j^{(ji)}}_{f(i|j)} + \underbrace{\gamma_u^{(uj)} \cdot \gamma_j^{(ju)}}_{f(u,j)}$$

This particular model is called *Factorizing Personalized Markov Chains* (FPMC)

Markov Chain models: FPMC

Note: terms cancel when contrasting examples

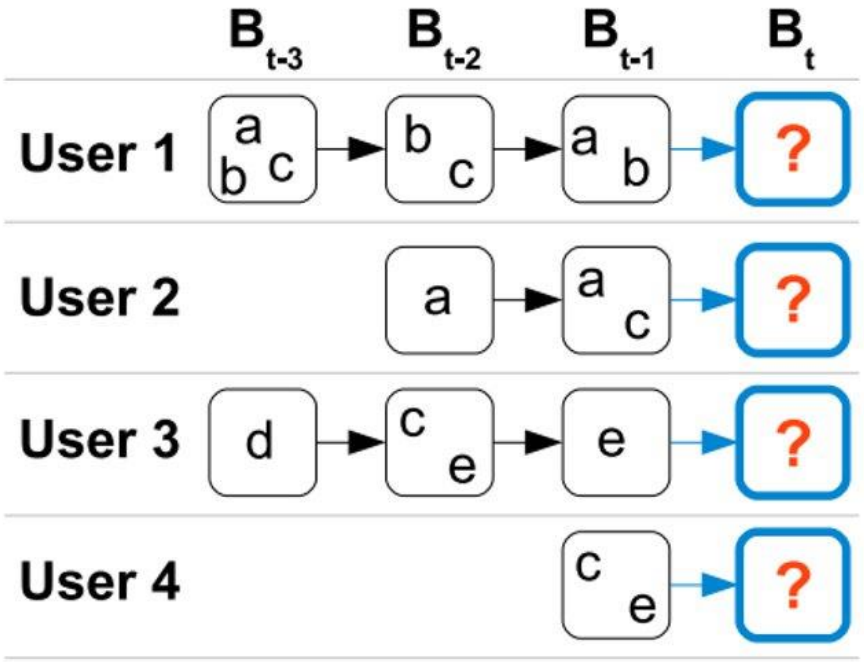
Markov Chain models: FPMC

Note: can be implemented using a factorization machine!

Factorizing Personalized Markov Chains for Next-Basket Recommendation

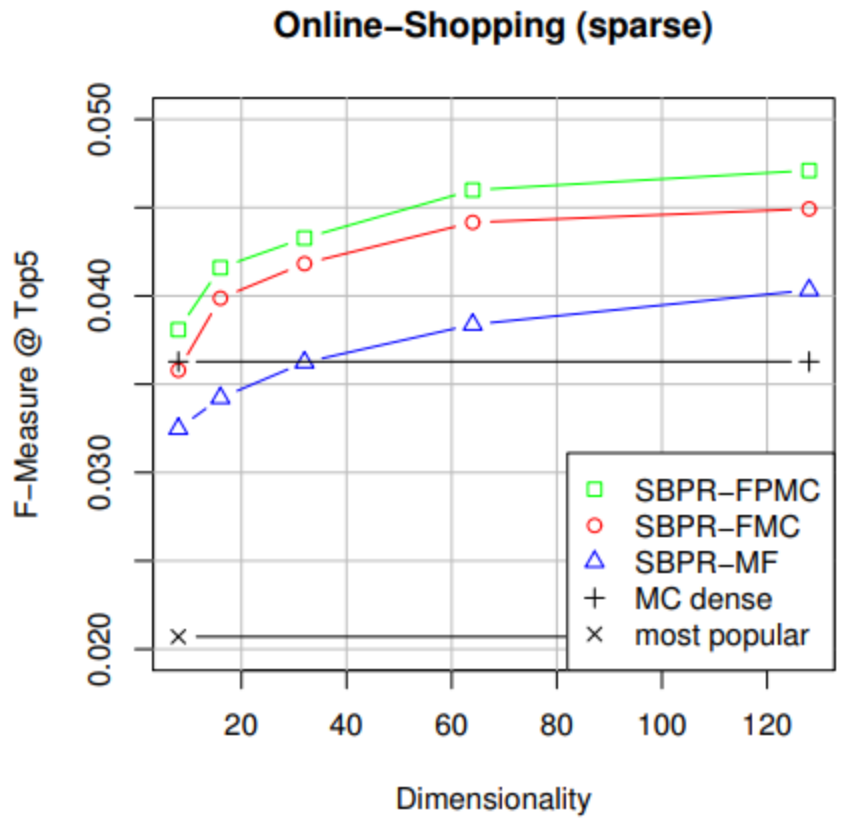
More from the original paper:

- Original paper actually looks at sequential *baskets* rather than sequential *items;* but this is mainly an implementation detail



Factorizing Personalized Markov Chains for Next-Basket Recommendation

F@5



FMC: not personalized

MF: personalized, but not sequentially-aware

Factorizing Personalized Markov Chains for Next-Basket Recommendation

Morals of the story:

- Can improve performance by modeling **third order** interactions between the user, the item, and the previous item
- This is simpler than temporal models – but makes a big assumption
- Given the blowup in the interaction space, this can be handled by **tensor decomposition** techniques

Web Mining and Recommender Systems

Case Studies

Case studies (will only cover one or two)

- Playlist prediction and POI recommendation: inner products versus distances
- Translation-based recommendation (sequential recommendation as a translation operation)
- Modeling user "expertise" over time

Personalized Ranking Metric Embedding for Next New POI Recommendation

Personalized Ranking Metric Embedding for Next New POI Recommendation

Shanshan Feng,[1] Xutao Li,[2] Yifeng Zeng,[3] Gao Cong,[2] Yeow Meng Chee,[4] Quan Yuan[2]

[1]Interdisciplinary Graduate School,
Nanyang Technological University, Singapore, sfeng003@e.ntu.edu.sg
[2]School of Computer Engineering, Nanyang Technological University, Singapore,
{lixutao@, gaocong@, qyuan1@e.}ntu.edu.sg
[3]School of Computing, Teesside University, UK, Y.Zeng@tees.ac.uk
[4]School of Physical and Mathematical Sciences,
Nanyang Technological University, Singapore, ymchee@ntu.edu.sg

Abstract

The rapidly growing of Location-based Social Networks (LBSNs) provides a vast amount of check-in data, which enables many services, e.g., point-of-interest (POI) recommendation. In this paper, we study the *next new* POI recommendation problem in which *new* POIs with respect to users' current location are to be recommended. The challenge lies in the difficulty in precisely learning users' sequential information and personalizing the recommendation model. To this end, we resort to the Metric Embedding method for the recommendation, which avoids drawbacks of the Matrix Factorization technique. We propose a personalized ranking metric embedding method (PRME) to model personalized check-in sequences. We further develop a PRME-G

mation of users' check-ins. The sequential behavior is important for POI recommendation because human movement exhibits sequential patterns [Ye *et al.*, 2013]. We verify users' sequential behavior in the analysis of two real-world datasets. Meanwhile, we observe that users often visit *new* POIs that they have not been visited before. In this paper, we focus on the *Next New* POI recommendation problem (simplified as N^2-POI recommendation), which is to recommend *new* POIs to be visited *next* given a user's current location.

The challenge of N^2-POI recommendation is to learn transitions of users' check-ins that are commonly represented by a first-order Markov chain model. Due to the sparse transition data, it is difficult to estimate the transition probability in Markov chain, especially for the unobserved transition. Factorized Personalized Markov Chain (FPMC) [Rendle *et al.*, 2010] method has been used to calculate the item transitions.

Factorizing Personalized Markov Chains for Next-Basket Recommendation

Goal: Can we build better sequential recommendation models by using the idea of **metric embeddings**

$$\gamma_u \cdot \gamma_i \quad \text{vs.} \quad d(\gamma_u, \gamma_i)$$

Personalized Ranking Metric Embedding for Next New POI Recommendation

Why would we expect this to work (or not)?

Personalized Ranking Metric Embedding for Next New POI Recommendation

Otherwise, goal is the same as the previous paper:

$$p_u(i|j)$$

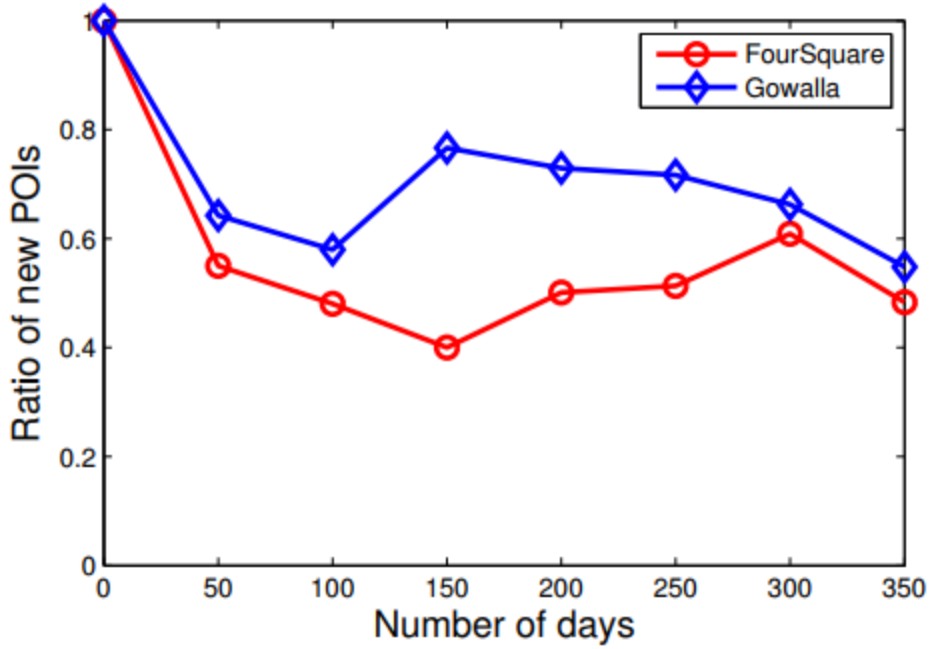
Personalized Ranking Metric Embedding for Next New POI Recommendation

Data

Dataset	#User	#POI	#Check-in	Time range
FourSquare	1917	2675	155365	08/2010-07/2011
Gowalla	4996	6871	245157	11/2009-10/2010

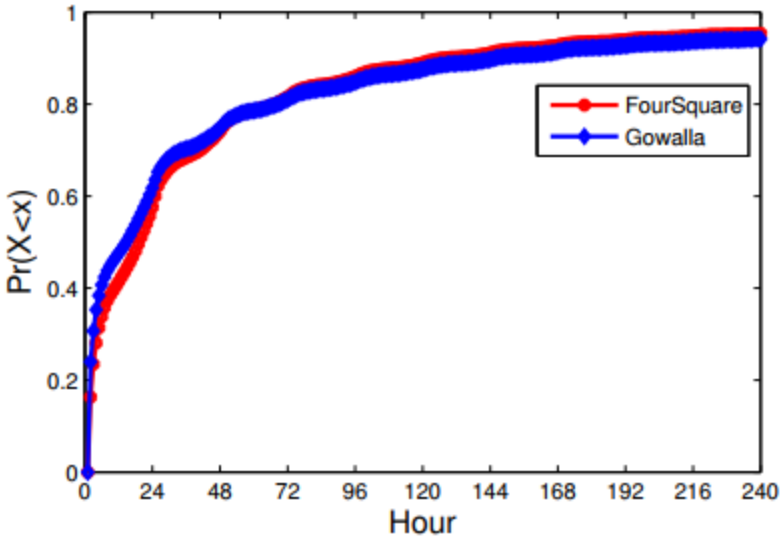
Personalized Ranking Metric Embedding for Next New POI Recommendation

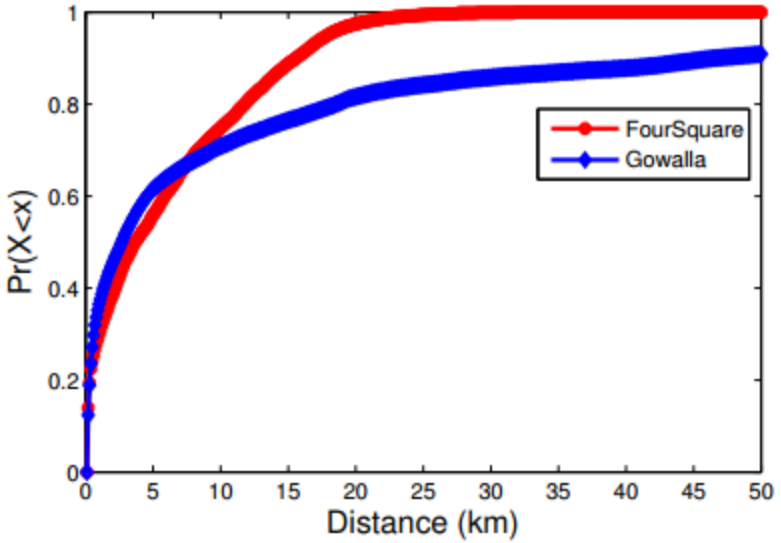
Qualitative analysis



Personalized Ranking Metric Embedding for Next New POI Recommendation

Qualitative analysis





Personalized Ranking Metric Embedding for Next New POI Recommendation

Basic model (not personalized)

$$\hat{P}(l_j|l_i) = \frac{e^{-||X(l_j)-X(l_i)||^2}}{Z(l_i)}$$

Personalized Ranking Metric Embedding for Next New POI Recommendation

Basic model (not personalized)

$$l_i >_{l^c} l_j \Leftrightarrow \hat{P}(l_i|l^c) > \hat{P}(l_j|l^c)$$

Personalized Ranking Metric Embedding for Next New POI Recommendation

Personalized version

$$\mathcal{D}_{u,l^c,l} = \alpha \mathcal{D}^P_{u,l} + (1-\alpha)\mathcal{D}^S_{l^c,l}$$

Personalized Ranking Metric Embedding for Next New POI Recommendation

Personalized version

$$\mathcal{D}_{u,l^c,l} = \begin{cases} \mathcal{D}^P_{u,l} & \text{if } \Delta(l, l^c) > \tau \\ \alpha \mathcal{D}^P_{u,l} + (1-\alpha)\mathcal{D}^S_{l^c,l} & \text{otherwise} \end{cases}$$

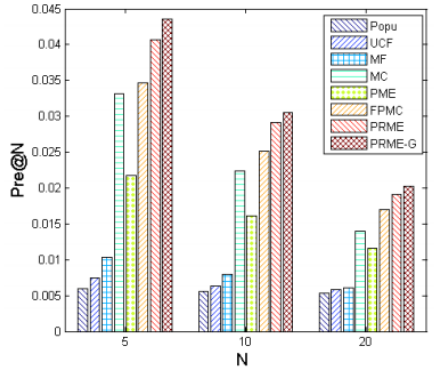
Personalized Ranking Metric Embedding for Next New POI Recommendation

Learning

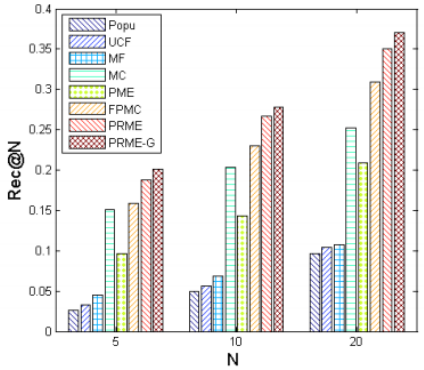
$$P(>_{u,l^c} |\Theta) = P\left((\mathcal{D}_{u,l^c,l_j} - \mathcal{D}_{u,l^c,l_i}) > 0|\Theta\right)$$
$$= \sigma(\mathcal{D}_{u,l^c,l_j} - \mathcal{D}_{u,l^c,l_i})$$

Personalized Ranking Metric Embedding for Next New POI Recommendation

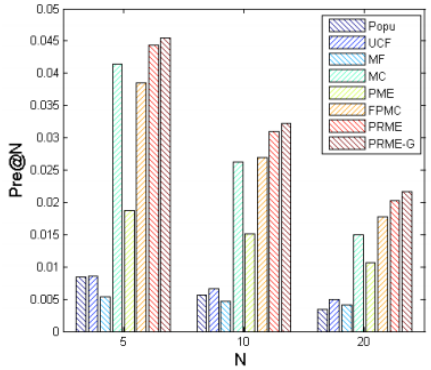
Results



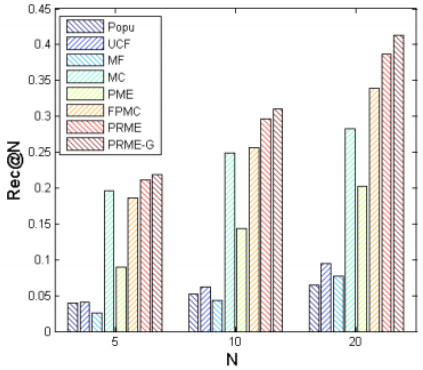
(a) Precision on FourSquare



(b) Recall on FourSquare



(c) Precision on Gowalla



(d) Recall on Gowalla

Personalized Ranking Metric Embedding for Next New POI Recommendation

Morals of the story:

- In some applications, **metric embeddings** might be better than inner products
- Examples could include geographical data, but also others (e.g. playlists?)

Web Mining and Recommender Systems

Playlist Prediction via Metric Embedding

Playlist prediction via Metric Embedding

Playlist Prediction via Metric Embedding

Shuo Chen
Cornell University
Dept. of Computer Science
Ithaca, NY, USA
shuochen@cs.cornell.edu

Joshua L. Moore
Cornell University
Dept. of Computer Science
Ithaca, NY, USA
jlmo@cs.cornell.edu

Douglas Turnbull
Ithaca College
Dept. of Computer Science
Ithaca, NY, USA
dturnbull@ithaca.edu

Thorsten Joachims
Cornell University
Dept. of Computer Science
Ithaca, NY, USA
tj@cs.cornell.edu

ABSTRACT

Digital storage of personal music collections and cloud-based music services (e.g. Pandora, Spotify) have fundamentally changed how music is consumed. In particular, automatically generated playlists have become an important mode of accessing large music collections. The key goal of automated playlist generation is to provide the user with a *coherent* listening experience. In this paper, we present Latent Markov Embedding (LME), a machine learning algorithm for generating such playlists. In analogy to matrix factorization methods for collaborative filtering, the algorithm does not require songs to be described by features a priori, but it learns a representation from example playlists. We formulate this problem as a regularized maximum-likelihood embedding of Markov chains in Euclidian space, and show how

addition, when using a cloud-based service like Rhapsody or Spotify, the consumer has instant on-demand access to millions of songs. This has created substantial interest in automatic playlist algorithms that can help consumers explore large collections of music. Companies like Apple and Pandora have developed successful commercial playlist algorithms, but relatively little is known about how these algorithms work and how well they perform in rigorous evaluations.

Despite the large commercial demand, comparably little scholarly work has been done on automated methods for playlist generation (e.g., [13, 4, 9, 11]), and the results to date indicate that it is far from trivial to operationally define what makes a playlist coherent. The most comprehensive study was done by [11]. Working under a model where

Playlist prediction via Metric Embedding

Goal: Build a recommender system that recommends sequences of songs

Idea: Might also use a metric embedding (consecutive songs should be “nearby” in some space)

Playlist prediction via Metric Embedding

Basic model:

$$\Pr(p^{[i]}|p^{[i-1]}) = \frac{e^{-||X(p^{[i]})-X(p^{[i-1]})||_2^2}}{\sum_{j=1}^{|S|} e^{-||X(s_j)-X(p^{[i-1]})||_2^2}}$$

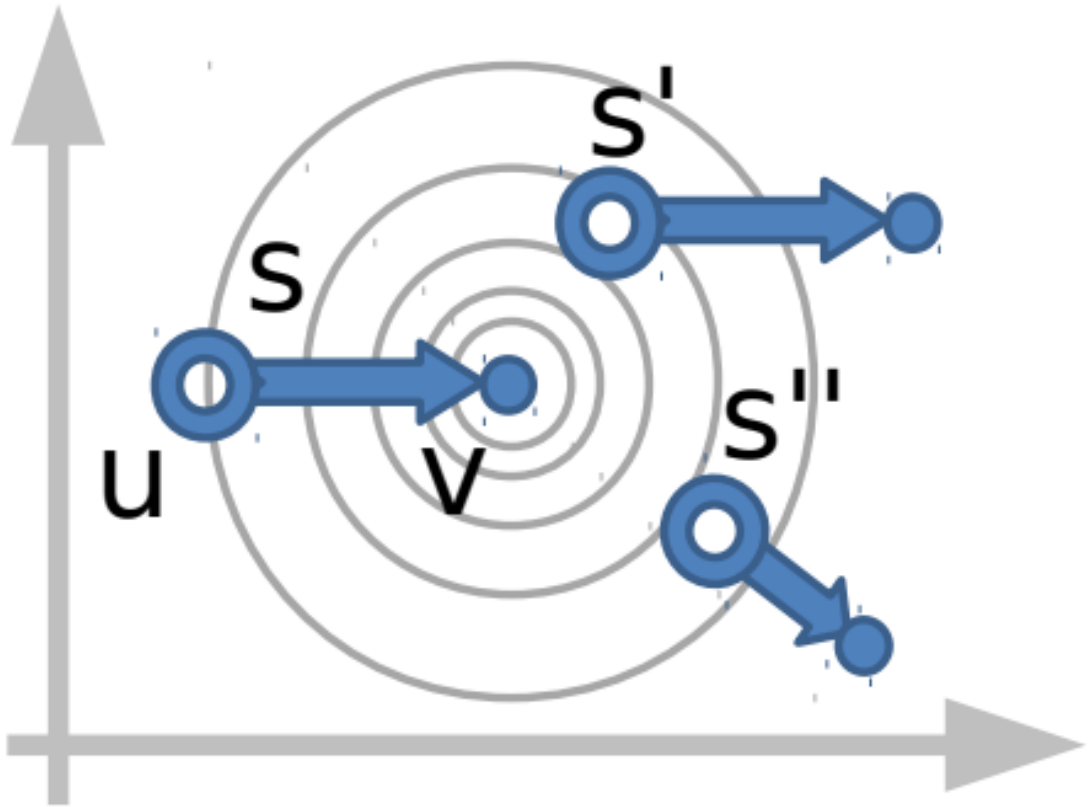
(compare with metric model from previous section)

Playlist prediction via Metric Embedding

Basic model (“single point”):

Playlist prediction via Metric Embedding

“Dual-point” model



$$||X(s) - X(s')||_2$$

$$\downarrow$$

$$||V(s) - U(s')||_2$$

Playlist prediction via Metric Embedding

Extensions:

- Popularity biases

$$\Pr(p^{[i]}|p^{[i-1]}) = \frac{e^{-\Delta(p^{[i]},p^{[i-1]})^2+b_i}}{\sum_j e^{-\Delta(s_j,p^{[i-1]})^2+b_j}}$$

Playlist prediction via Metric Embedding

Extensions:

- Personalization

$$\Pr(p^{[i]}|p^{[i-1]},u)=\frac{e^{-\Delta(p^{[i]},p^{[i-1]})^2+A(p^{[i]})^T B(u)}}{\sum_j e^{-\Delta(s_j,p^{[i-1]})^2+A(s_j)^T B(u)}}$$

Playlist prediction via Metric Embedding

Extensions:

- Semantic Tags

$$\Pr(X(s)|T(s)) = \mathcal{N}\left(\frac{1}{|T(s)|}\sum_{t\in T(s)} M(t),\ \frac{1}{2\lambda} I_d\right)$$

Playlist prediction via Metric Embedding

Extensions:

- Observable Features

$$\Pr(p^{[i]}|p^{[i-1]}) = \frac{e^{-\Delta(p^{[i]},p^{[i-1]})^2 + O(p^{[i]})^T W O(p^{[i-1]})}}{\sum_j e^{-\Delta(s_j,p^{[i-1]})^2 + O(s_j)^T W O(p^{[i-1]})}}$$

Playlist prediction via Metric Embedding

Experiments:

Yes.com playlists

- Dec 2010 – May 2011

"Small" dataset:

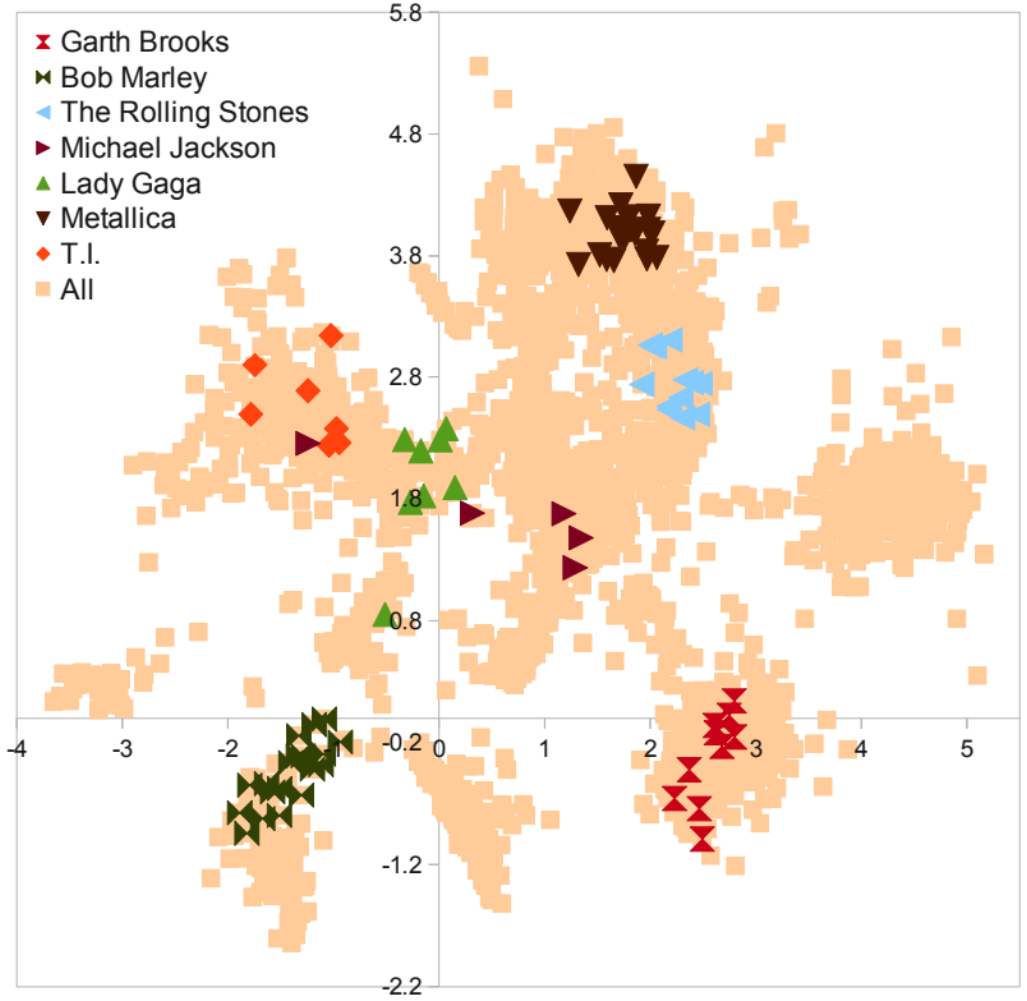
- 3,168 songs
- 134,431 playlists + 1,191,279 transitions

"Large" dataset

- 9,775 songs
- 172,510 playlists + 1,602,079 transitions

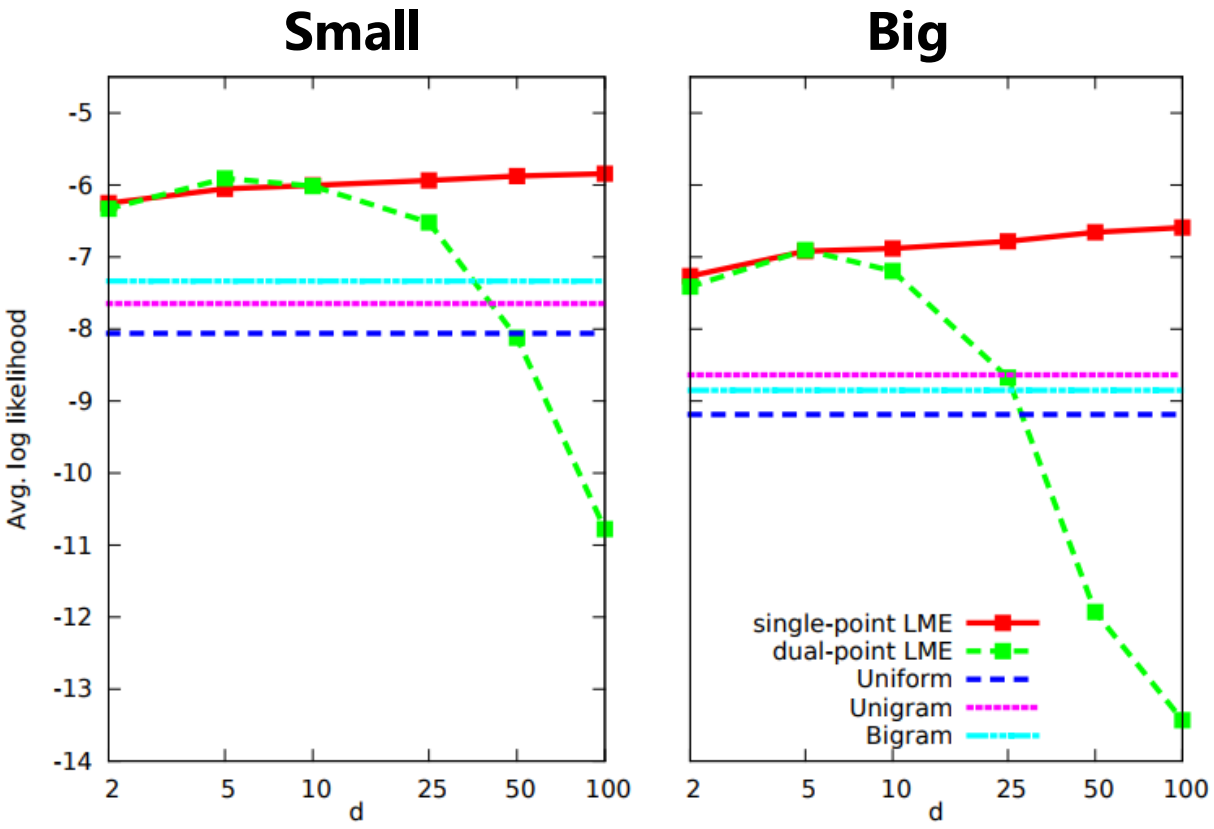
Playlist prediction via Metric Embedding

Experiments:



Playlist prediction via Metric Embedding

Experiments:



Playlist prediction via Metric Embedding

Morals of the story:

- Metric assumption works well in settings other than “geographical” data!
- However, they require some modifications in order to work well (e.g. “start points” and “end points”)
- Effective combination of latent + observed features, as well as metric + inner-product models

Web Mining and Recommender Systems

Temporal Modeling of Reviewer Expertise

Why do Americans like Pale Ale?

Hop cone

Lupulin



American Pale Ales:

- Hopsecutioner
- Hoptimus Prime
- Smooth Hoperator
- Red Hoptober
- Hoppy ending
- Hoptopus
- Hopsickle
- Tricerahops

Lupulin threshold shift:

People become accustomed to hops over time,
and can recognize more subtle flavors

Users and products evolve over time

"Classics" are rated better (Koren, 2010); new products cause users to change focus (Koller & Malouf, 2007)

Age of the **product**

?

(development) of the **user**

Users influence each other (Ma et al., 2011); communities shift over time (Xiong et al., 2010)

Age (zeitgeist) of the **community**

How can we effectively characterize **acquired tastes** or **expertise**?

Data

ratebeer

3M reviews, **100K** beers, **40K** users

Beeradvocate

1.5M reviews, **60K** beers, **30K** users



2M reviews, **500K** wines, **45K** users

Models of user and community evolution

Replace the ‘standard’ model

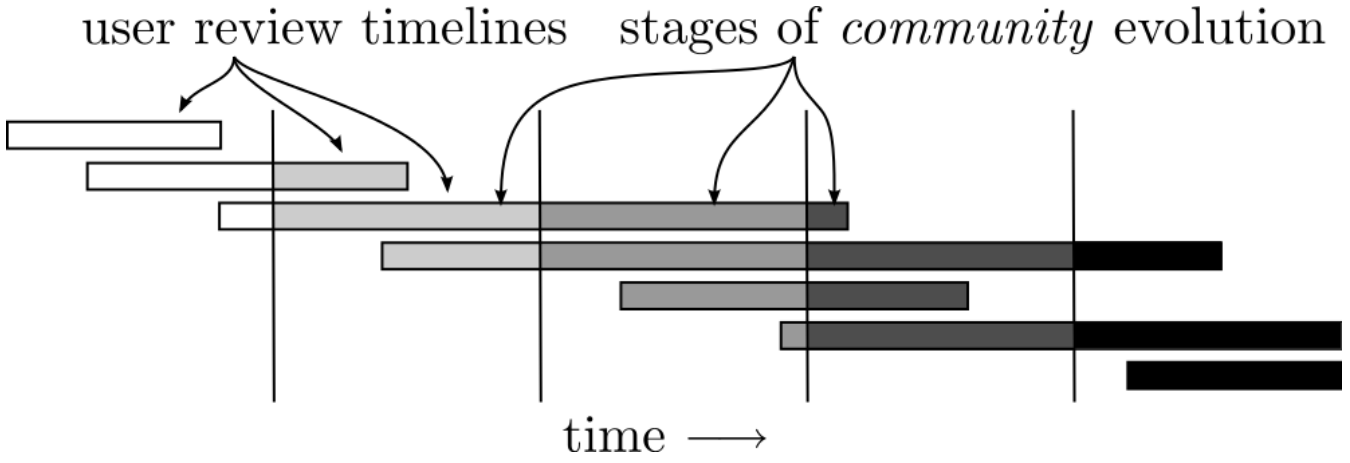
$$rec(u,i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

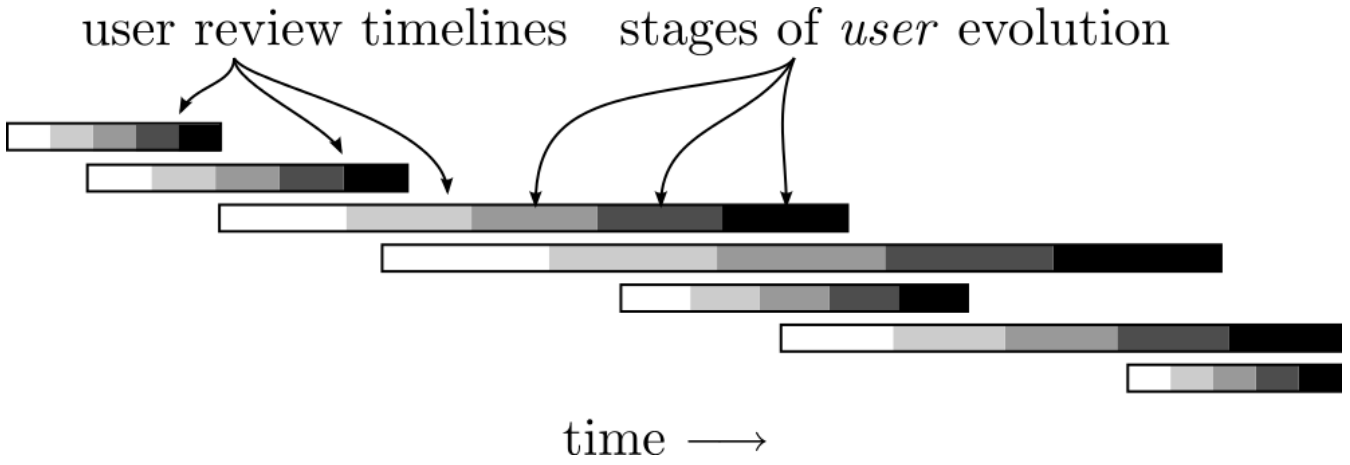
With one whose parameters change as a function of time (t)

$$rec_t(u,i) = \alpha(t) + \beta_u(t) + \beta_i(t) + \gamma_u(t) \cdot \gamma_i(t)$$

How we define *t* determines what type of evolution we model

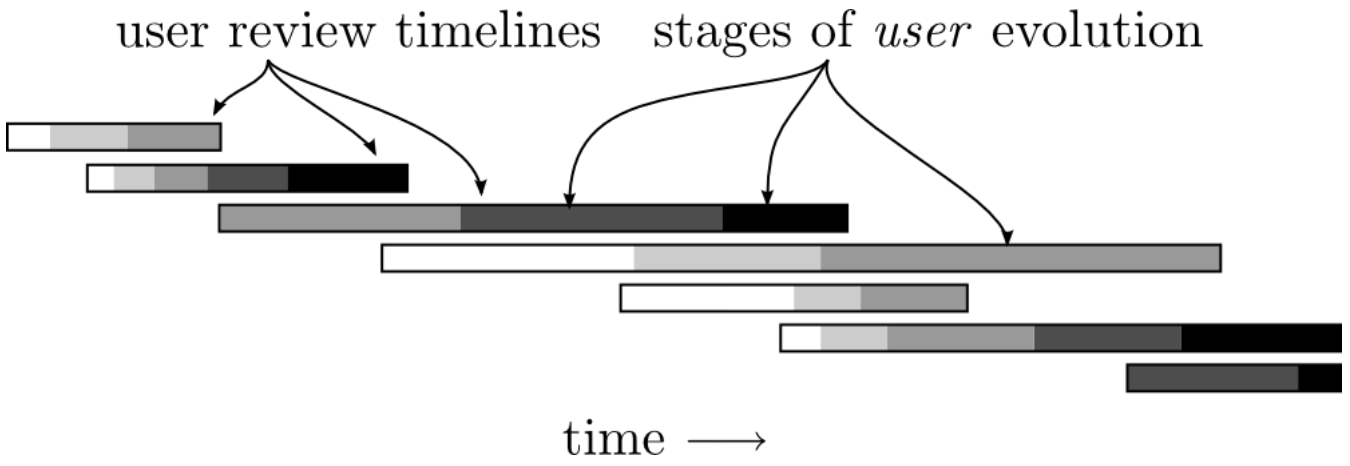
Models of user and community evolution





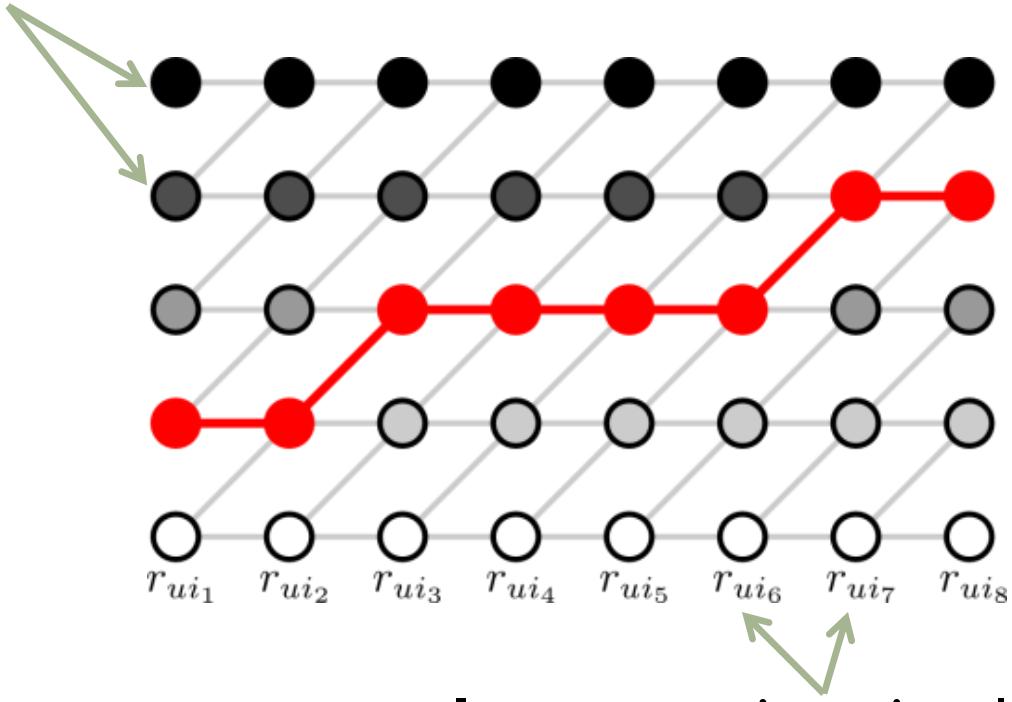
Models of user and community evolution

Observation:
People evolve and develop at different rates.
We must **learn** the rate of development for each user



Models of user and community evolution

rows: models of increasingly "experienced" users



columns: review timeline for one user

Each user's evolution can be thought of as a **monotonic** path through a graph

Optimization problem & fitting

Model:

$$rec_{e_{u,i}}(u,i) = \alpha(e) + \beta_u(e) + \beta_i(e) + \gamma_u(e) \cdot \gamma_i(e)$$

experience at time of review

offset, bias (user/item), and latent factors

Optimization problem:

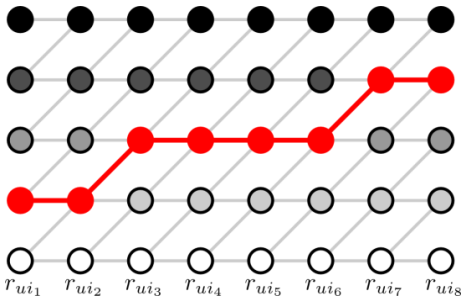
$$\arg\min_{\Theta,\mathcal{E}} \frac{1}{|\mathcal{T}|} \sum_{r_{u,i} \in \mathcal{T}} \underbrace{\left(rec_{e_{u,i}}(u,i) - r_{u,i}\right)^2}_{\text{rating error}} + \underbrace{\Omega(\Theta)}_{\text{regularizer}}$$

model & experience parameters

smoothness & l2 regularity

Optimization problem & fitting

Repeat steps (1) and (2) until convergence:



(solved using dynamic programming)

Step 1: fit expertise progression

$$\arg\min_{\Theta} \frac{1}{|\mathcal{T}|} \sum_{r_{u,i} \in \mathcal{T}} (rec_e(u,i) - r_{u,i})^2 + \Omega(\Theta)$$

solved via gradient ascent using L-BFGS
(see e.g. Koren & Bell, 2011)

Step 2: fit rating models for each expertise level

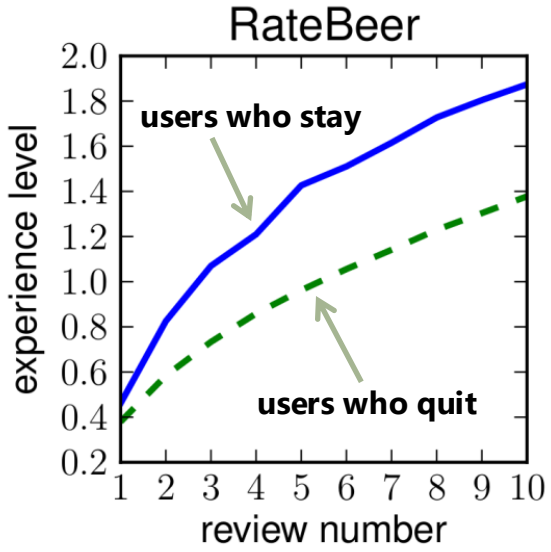
Outcomes – applications

Rating prediction:

- Beer: 6% improvement over state-of-the-art
- Wine: 13% improvement
- Movies (Amazon): 23% improvement

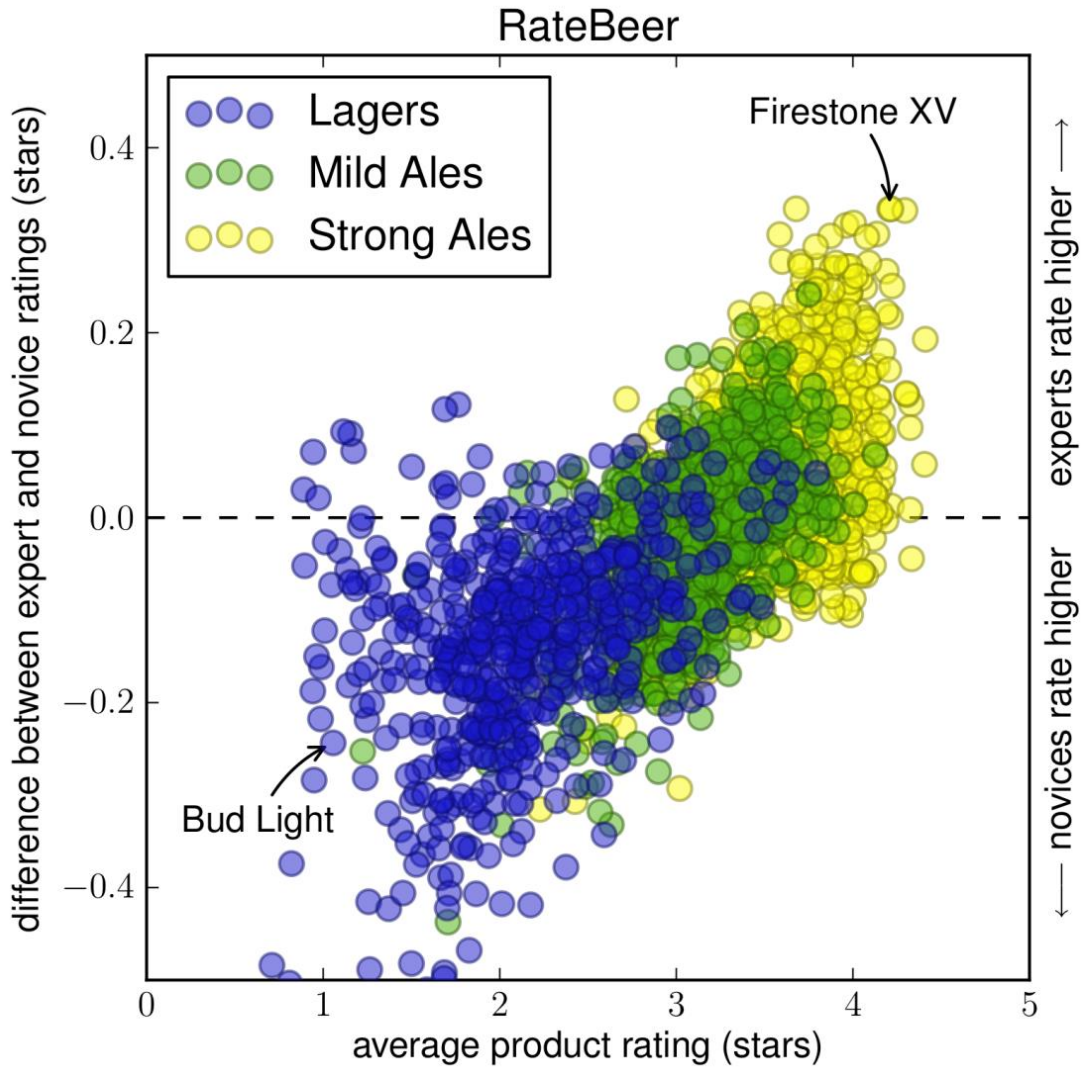
User retention:

What happens to users who **fail** to acquire taste for a product?



Users who acquire tastes slowly are more likely to quit the community

Outcomes – understanding



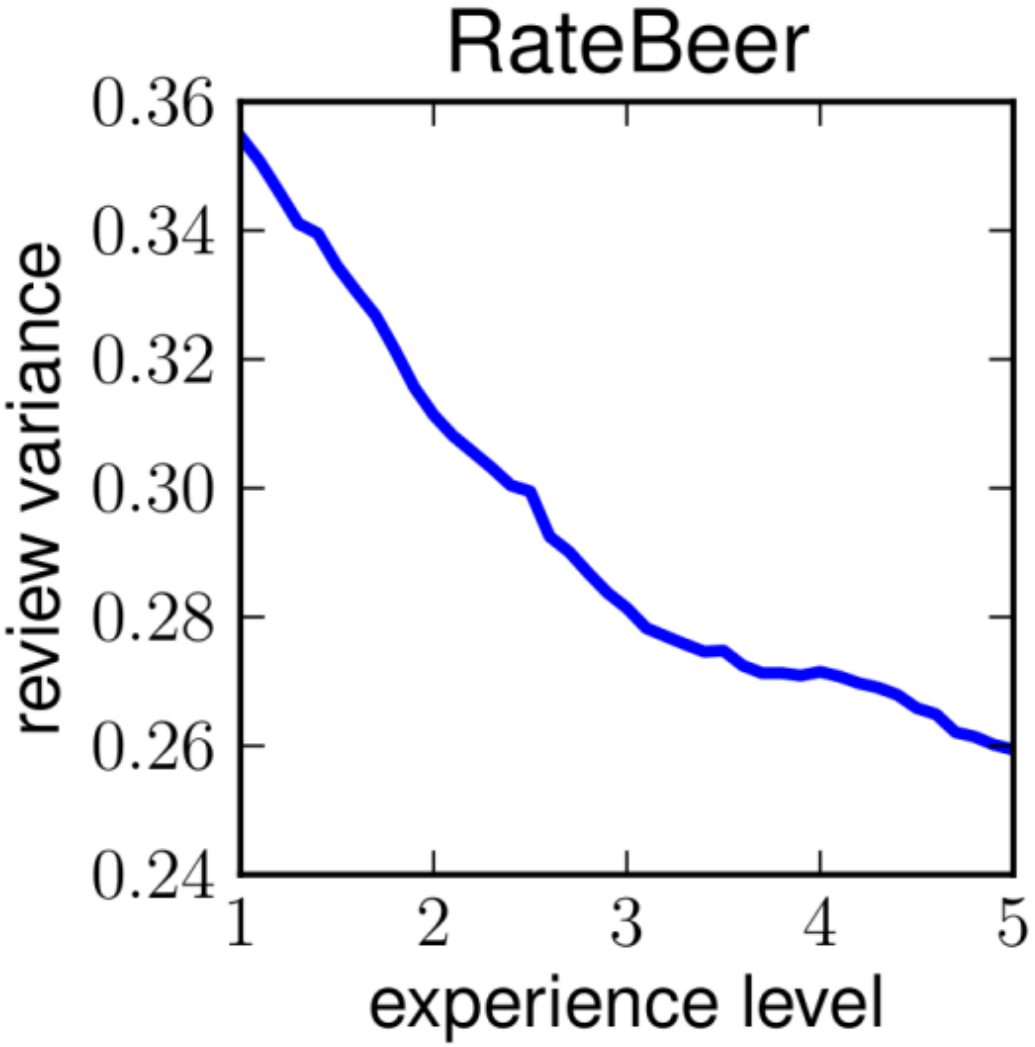
1. Entire categories of products tend to be preferred by experts or beginners

2. Experts rate the top products more generously, and the bottom products more harshly

Outcomes – understanding

What are experts?
Predictability and agreement are two necessary conditions (Einhorn, 1974)

Experts are more predictable than beginners. They are also more inclined to agree with each other (right).



Web Mining and Recommender Systems

Translation-based recommendation

Translation-based Recommendation

Translation-based Recommendation

Ruining He
UC San Diego
r4he@cs.ucsd.edu

Wang-Cheng Kang
UC San Diego
wckang@eng.ucsd.edu

Julian McAuley
UC San Diego
jmcauley@cs.ucsd.edu

ABSTRACT

Modeling the complex interactions between users and items as well as amongst items themselves is at the core of designing successful recommender systems. One classical setting is predicting users' personalized sequential behavior (or 'next-item' recommendation), where the challenges mainly lie in modeling 'third-order' interactions between a user, her previously visited item(s), and the next item to consume. Existing methods typically decompose these higher-order interactions into a combination of *pairwise* relationships, by way of which user preferences (user-item interactions) and sequential patterns (item-item interactions) are captured by separate components. In this paper, we propose a unified method, *TransRec*, to model such third-order relationships for large-scale sequential prediction. Methodologically, we embed items into a 'transition space' where users are modeled as *translation* vectors operating on item sequences. Empirically, this approach outperforms the state-of-the-art on a wide spectrum of real-world datasets. Data and code are available at https://sites.google.com/a/eng.ucsd.edu/ruining-he/.

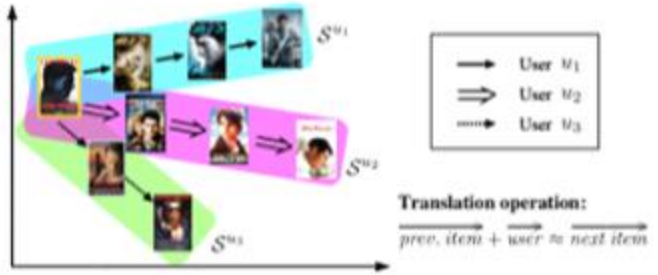


Figure 1: *TransRec* as a sequential model: Items (movies) are embedded into a 'transition space' where each user is modeled by a *translation* vector. The transition of a user from one item to another is captured by a user-specific translation operation. Here we demonstrate the historical sequences S^{u_1}, S^{u_2}, and S^{u_3} of three users. Given the same starting point, the movie *Mission: Impossible I*, u_1 went on to watch the whole series, u_2 continued to watch drama movies by Tom Cruise, and u_3 switched to similar action movies.

1 INTRODUCTION

Modeling and predicting the *interactions* between users and items, as well as the *relationships* amongst the items themselves are the main tasks of recommender systems. For instance, in order to predict *sequential* user actions like the next product to purchase, movie to watch, or place to visit, it is essential (and challenging!) to model the *third-order* interactions between a user (u), the item(s)

FPMC models third-order relationships between u, i, and j by the *summation* of two pairwise relationships: one for the compatibility between u and the next item j, and another for the sequential continuity between the previous item i and the next item j. Ultimately, this is a combination of MF and MC (see Section 3.5 for details).

Translation-based Recommendation

Goal: (e.g) which movie is this user going to watch next?



Want models that consider

- characteristics/preferences of each user
- local context, i.e., the last consumed item(s)

Translation-based Recommendation

Compare: Factorized Personalized Markov Chains (earlier today)

$$Prob(j \mid u, i) \propto \underbrace{\langle \vec{M}_u, \vec{N}_j \rangle}_{\text{user preference}} + \underbrace{\langle \vec{P}_i, \vec{Q}_j \rangle}_{\text{local context}}$$

Translation-based Recommendation

Compare: Personalized Ranking Metric Embedding (earlier today)

$$Prob(j \mid u, i) \propto -\left(\alpha \cdot \|\vec{M}_u - \vec{N}_j\|_2^2 + (1-\alpha) \cdot \|\vec{P}_i - \vec{P}_j\|_2^2\right)$$

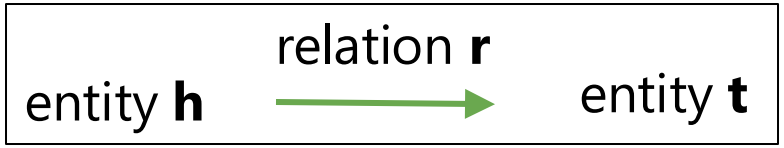
an additional hyperpara. to balance the two components

Translation-based Recommendation

Detour: Translation models in Knowledge Bases

Data: entities; links (multiple types of relationships)
State-of-the-art method: *'relationships as translations'*
Goal: Predict unseen links

Training example:



Basic idea:

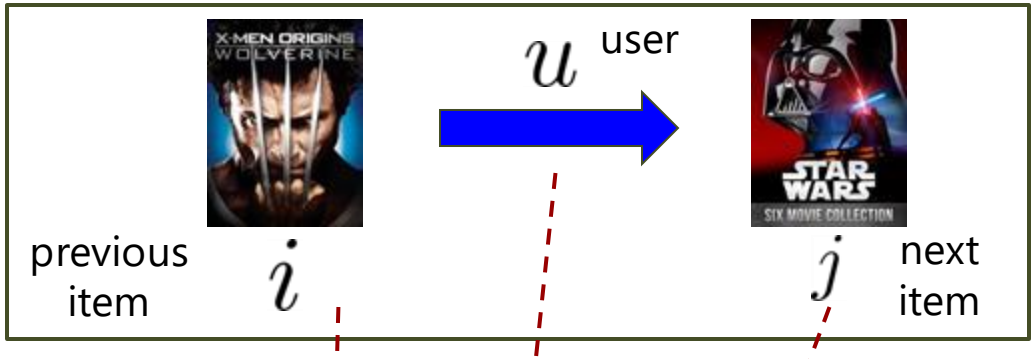
$$\vec{\mathbf{h}} + \vec{\mathbf{r}} \approx \vec{\mathbf{t}}$$

E.g. [Bordes *et al.*, 2013], [Wang *et al.*, 2014], [Lin *et al.*, 2015]

Translation-based Recommendation

Embedding space { Items as *points*
Users as *translation vectors*

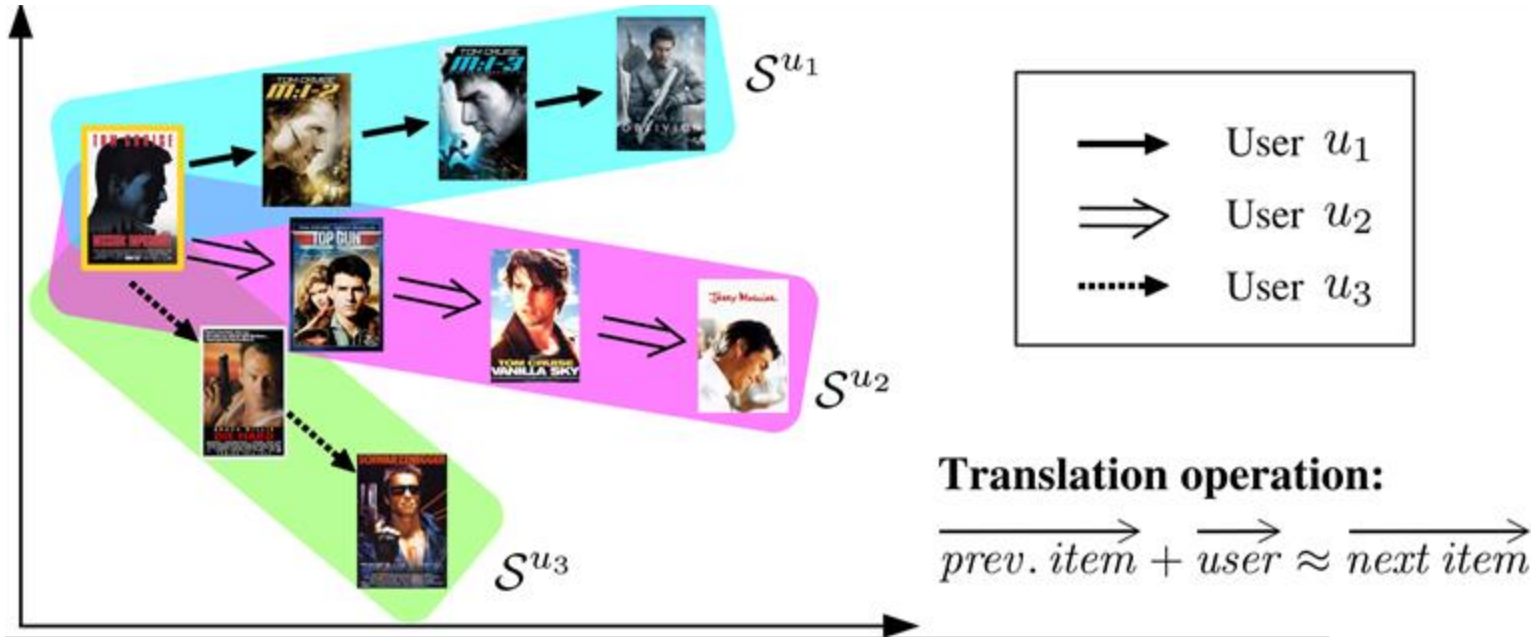
Training triplet:



Objective: $\vec{\gamma}_i + \vec{t}_u \approx \vec{\gamma}_j$

Translation-based Recommendation

Embedding space { Items as *points*
Users as *translation vectors*



Translation-based Recommendation

$$Prob(j|u,i) \propto \beta_j - d(\vec{\gamma}_i + \vec{t}_u, \vec{\gamma}_j)$$

bias $\quad \mathcal{L}_1/\mathcal{L}_2/\cdots$

- Benefit from using metric embeddings
- Model (u, i, j) with a single component
- Recommendations can be made by a simple NN search

Translation-based Recommendation

$$\widehat{\Theta} = \arg\max_{\Theta} \ln \prod_{u \in \mathcal{U}} \prod_{j \in \mathcal{S}^u} \prod_{j' \notin \mathcal{S}^u} Prob(j >_{u,i} j' | \Theta) \, Prob(\Theta)$$

$$= \arg\max_{\Theta} \sum_{u \in \mathcal{U}} \sum_{j \in \mathcal{S}^u} \sum_{j' \notin \mathcal{S}^u} \ln \sigma(\widehat{p}_{u,i,j} - \widehat{p}_{u,i,j'}) - \Omega(\Theta),$$

Translation-based Recommendation

Works well with…

Doesn’t work well with…

Overview

Morals of the story:

- Today we looked at two main ideas that extend the recommender systems we saw in class:

1. **Sequential Recommendation:** Most of the dynamics due to time can be captured purely by knowing the *sequence* of items

2. **Metric Recommendation:** In some settings, using inner products may not be the correct assumption

Web Mining and Recommender Systems

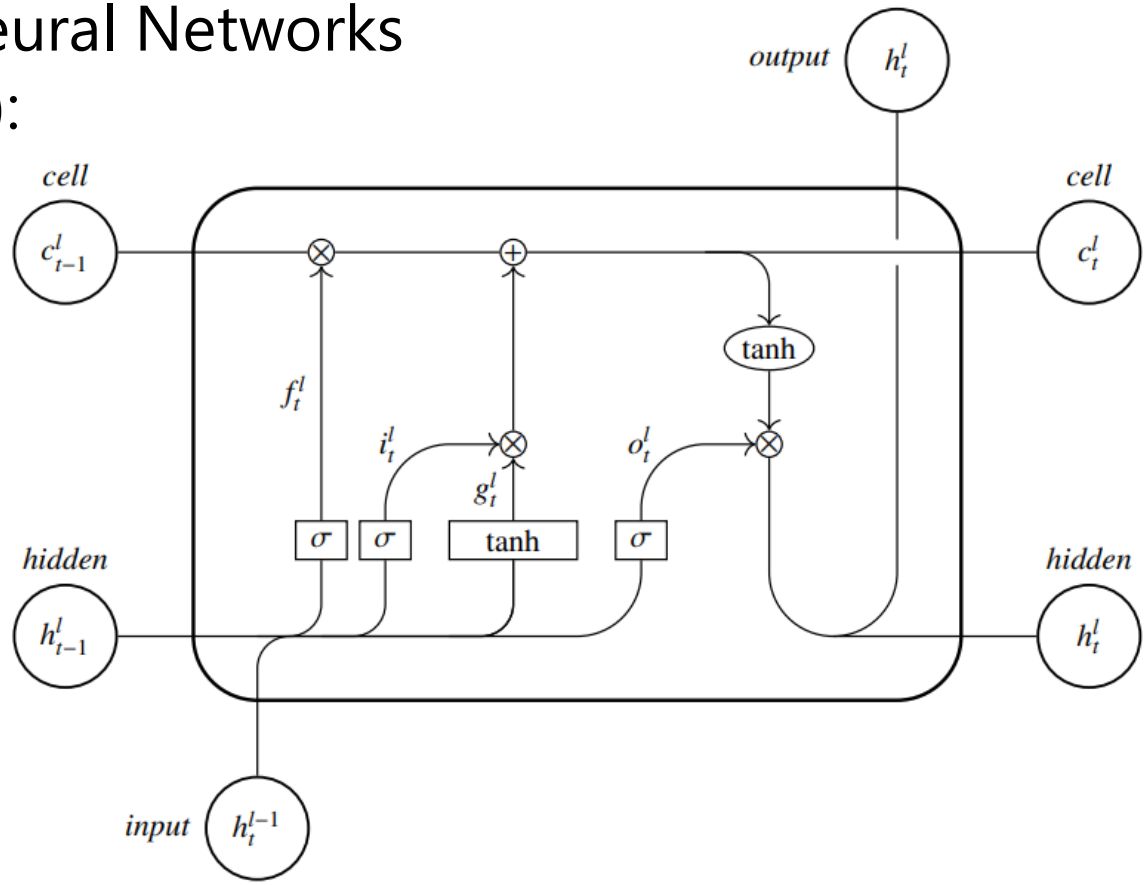
A few other approaches to temporal recommendation

Recurrent Neural Networks

- Markov Chains are a "traditional" model for sequence prediction
- Can more advanced sequence prediction models (e.g. Recurrent Neural Networks) be applied in the context of recommendation?

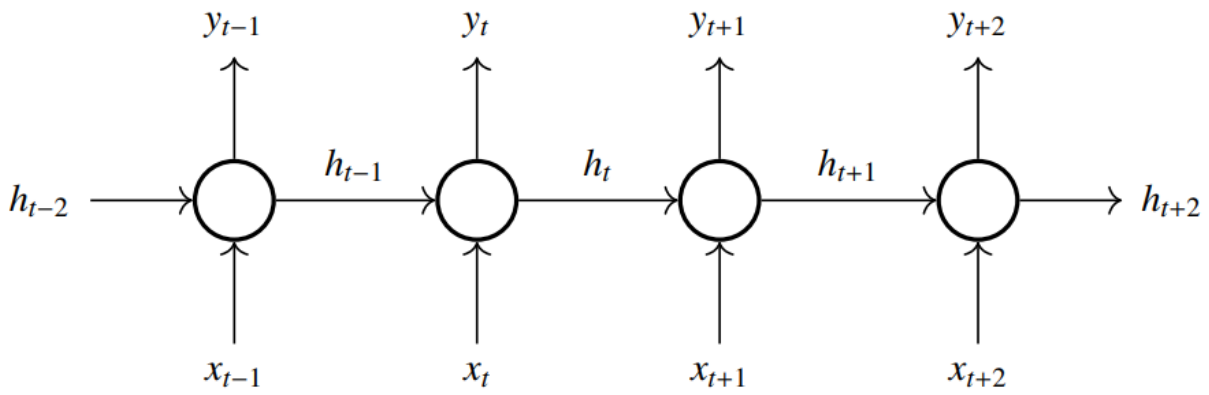
Recurrent Neural Networks

E.g. Recurrent Neural Networks (LSTM, GRU, etc.):



Recurrent Neural Networks

E.g. Recurrent Neural Networks
(LSTM, GRU, etc.):



Recurrent Neural Networks

These types of sequential model can (potentially) capture longer-term dynamics in data, compared to what's available in a Markov-Chain model

- Recommendation problems could have longer-term semantics, e.g. as users revisit items across sessions, or the next item they visit is relevant to a combination of recent items
- Could also be very expensive – often simpler approaches are favorable just because we have too little data to train complex models

Recurrent Neural Networks

Such models can be "user-free" (recall FISM, item2vec, etc.)

- Since the network can potentially store "state" about a user across a large number of steps, we don't need to store a representation of a user
- At inference time, just feed the recent interaction history into the model, and it will predict the next interaction
- (relatively easy to deploy!)

Recurrent Neural Networks

Lots of recent state-of-the-art models come from NLP!

- Models like item2vec (word2vec), LTSMs, GRUs etc. have famously been used for sequence modeling tasks in NLP
- Such models can be adapted to build recommenders, and often represent the current state-of-the-art
- See e.g. models based on Transformers, BERT, etc.
- More in textbook!